Université de Reims Champagne-Ardenne
Master 2 Calcul Haute Performance et Simulation (CHPS)

Compte-rendu de TP

Cours 6 : DevOps pour le HPC, séance en pédagogie inversée

Du dépôt Git au cluster : intégration continue, Slurm et Spack

carnet de bord des missions 1 à 5,
projet `mpiflow`, tag `seance-inversee`

Étudiant(s) : Sylvain ROWE
Formation : M2 CHPS
UE : CHPS0902
Enseignant : Luiz Angelo Steffenel
Date : 25 octobre 2026

Reste à faire avant le rendu

Cette page disparaît d'elle-même quand les deux scripts ci-dessous ont produit leurs preuves ; les mentions [À compléter : ...] sont alors remplies.

1. Sur votre machine, depuis le dépôt (branche `main`) : `GITLAB_TOKEN=glpat-... python3 scripts/gitlab_evidence.py`
2. Sur le nœud de connexion du cluster GCP, depuis le dépôt cloné dans `/home` : `bash scripts/gcp_evidence.sh`
3. Recompiler (`latexmk -g` et `python3 scripts/tex2md.py`, faits par les scripts si LaTeX et pandoc sont installés), commiter `logs/`, `spack/`, le PDF et le Markdown, puis `git tag -f seance-inversee && git push -f origin seance-inversee`.
4. `terraform destroy` et vérification de la console GCP.

1 Introduction

Un code de calcul ne vit pas seulement sur le poste du développeur : il est compilé, testé, empaqueté puis exécuté sur un cluster partagé, où tout passe par l'ordonnanceur Slurm et où l'environnement logiciel doit être identique sur chaque nœud. Le DevOps appliqué au HPC automatise cette chaîne, mais ses outils ne se transposent pas tels quels depuis le cloud : « CI $\neq$ exécution cluster ».

Cette séance en pédagogie inversée consistait à le vérifier par l'expérience, en quatre missions : faire exécuter par une autre machine la compilation et le test d'un code MPI (mission 1), conditionner les jobs à des événements et protéger les secrets (mission 2), faire franchir au pipeline la frontière entre le runner CI et Slurm (mission 3), et versionner l'environnement logiciel avec Spack (mission 4). Ce carnet de bord constitue la mission 5 : une section par mission, avec les réponses aux questions de cadrage, les preuves d'exécution, les erreurs rencontrées et un bilan, puis un auto-diagnostic.

Pour chaque question, nous avons cherché une réponse fondée sur une expérience plutôt que sur les diapositives : plusieurs réponses en sont sorties nuancées, voire contraires à notre intuition de départ.

2 Matériels et méthodes

2.1 Environnement

Pour pouvoir casser librement les choses (faire échouer un test, saturer une file, simuler la panne d'un nœud), nous avons d'abord reproduit chaque mission sur un poste Linux, avec un petit cluster Slurm complet et un émulateur de GitLab CI (tableau 1). Le mini-cluster comprend le contrôleur, la base de comptabilité (sans laquelle `sacct` ne fonctionne pas) et deux nœuds de 2 CPU, comme deux VM `e2-standard-2`. Les mêmes fichiers sont ensuite exécutés sur GitLab.com et sur le cluster GCP déployé par Terraform au Cours 5.

Tableau 1. Environnements d'exécution.

Composant	Version et configuration	Missions
Poste de reproduction	Ubuntu 24.04, 2 vCPU, 8 Go ; GCC 13.3, CMake 3.28, Open MPI 4.1.6	toutes
Analyse statique	cppcheck 2.13, shellcheck 0.9, actionlint 1.7.12	2, 3
Émulateur GitLab CI	`gitlab-ci-local` 4.76.1 : validation par le schéma officiel, évaluation des `rules`, exécution des jobs	1, 2
Mini-cluster Slurm	Slurm 23.11.4 : `slurmctld`, `slurmdbd` et MariaDB, deux `slurmd` (`node1`, `node2`, 2 CPU), partition `debug`, utilisateur sans privilèges	3
Spack	1.2.2 (tag de version), dépôt de paquets `builtin`, solveur clingo 5.8	4
GitLab.com	runners partagés, projet [À compléter : URL du dépôt]	1 à 3
Google Cloud	cluster Slurm du Cours 5 : [À compléter : type de VM, partition]	3, 4

2.2 Le dépôt `mpiflow`

Le dépôt de travail (tableau 2) contient un code de test MPI, son build CMake et tous les fichiers produits pendant les missions. Le pipeline final est résumé par la figure 1 ; le fichier complet est donné en annexe A.

Tableau 2. Contenu du dépôt.

Fichier	Rôle	Mission
`tests/test_mpi.c`	test MPI : Bcast, Allreduce, taille du communicateur	1
`CMakeLists.txt`, `tests/CMakeLists.txt`	build CMake, tests CTest à 1 et 4 rangs	1
`.gitlab-ci.yml`	pipeline : build, test, lint, package, secrets, Slurm	1 à 3
`.github/workflows/ci.yml`	équivalent GitHub Actions	1, 2
`scripts/fake_docker_login.sh`	simulation de `docker login --password-stdin`	2
`slurm/job_ci.sh`	job Slurm : 2 nœuds, MPI, `SUCCESS`	3
`slurm/submit_and_poll.sh`	soumission, suivi par `sacct`, verdict	3
`slurm/ci_remote.sh`	variante SSH : build et soumission à distance	3
`spack/spack.yaml`, `spack/spack.lock`	environnement Spack versionné	4
`logs/`	sorties brutes citées dans ce document	–

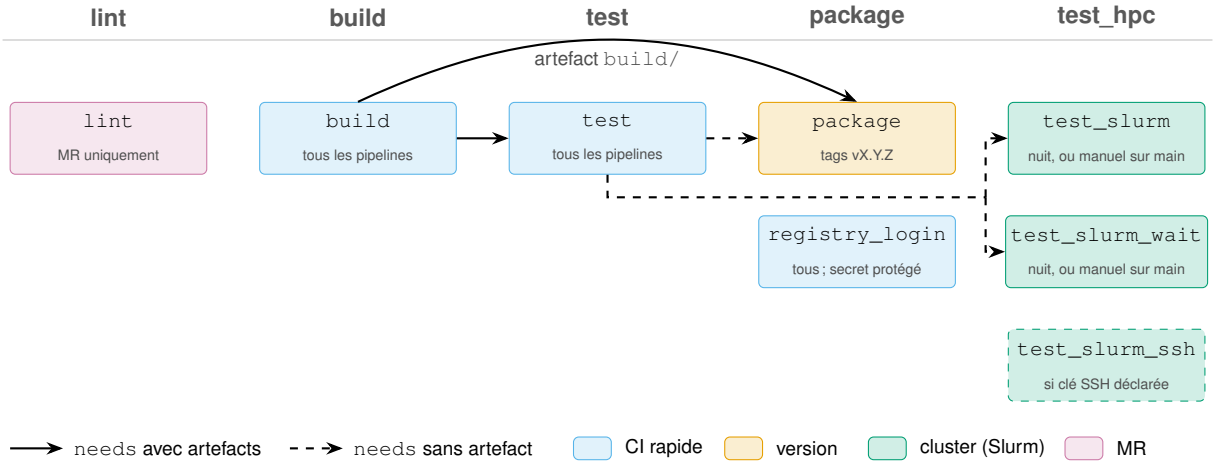


Figure 1. Pipeline final. Chaque colonne est un stage ; les flèches représentent les dépendances `needs`, sous chaque job figure la condition qui le crée. `lint`, `registry_login` et `test_slurm_ssh` ont `needs: []` et démarrent dès la création du pipeline. Les jobs Slurm attendent `test` sans récupérer son binaire, compilé pour l'image Debian et non pour le cluster.

2.3 Démarche et preuves

Chaque commande citée a été exécutée à travers un petit enregistreur qui écrit la commande, sa sortie complète et son code retour dans un fichier de `logs/`. Les listings de ce document incluent ces fichiers tels quels ; les rares coupes sont signalées par `[...]`. Les preuves produites sur GitLab.com et sur le cluster GCP sont collectées de la même façon, par `scripts/gitlab_evidence.py` (API de GitLab) et `scripts/gcp_evidence.sh` (sur le nœud de connexion), dans `logs/gitlab/` et `logs/gcp/`.

3 Mission 1 : un pipeline minimal

3.1 Réalisation

Le test `tests/test_mpi.c` (annexe C) vérifie trois choses : la valeur 42 diffusée par `MPI_Bcast` arrive sur tous les rangs, tous les rangs participent (`MPI_Allreduce` de la somme des rangs) et, si la variable `EXPECTED_RANKS` est définie, `MPI_COMM_WORLD` a bien cette taille. Le rang 0 affiche aussi le nœud de chaque rang, ce qui servira de preuve d'exécution multi-nœuds en mission 3. Les vérifications sont explicites et renvoient un code d'erreur, au lieu d'utiliser `assert` comme le test des diapositives : nous avons constaté que cette assertion disparaît en

compilation `Release` (section 3.4).

CTest déclare deux tests, à 1 et 4 rangs, à partir des variables `MPIEXEC_*` de `FindMPI`. Le pipeline (listing 1) enchaîne deux jobs dans l'image `gcc:13-bookworm` : `build` publie `build/` comme artefact, `test` le récupère via `needs: [build]`, lance `ctest` (le rapport JUnit s'affiche dans GitLab) puis `mpirun -np 4`.

```
47 build:
48   stage: build
49   extends: [.apt_mpi, .every_pipeline]
50   script:
51     - cmake -B build -DCMAKE_BUILD_TYPE=Release
52         -DMPIEXEC_PREFLAGS="--allow-run-as-root;--oversubscribe"
53     - cmake --build build -j"$(nproc)"
54   artifacts:
55     paths: [build/]
56     expire_in: 1 week
57
58 test:
59   stage: test
60   extends: [.apt_mpi, .every_pipeline]
61   needs: [build]                        # démarre dès la fin de build, récupère build/
62   script:
63     - cd build
64     - ctest --output-on-failure --output-junit junit.xml
65     # root dans le conteneur + 2 à 4 cœurs sur le runner : les deux options sont requises
66     - mpirun --allow-run-as-root --oversubscribe -np 4 tests/test_mpi
67   artifacts:
68     when: always                        # le rapport JUnit est utile surtout en cas d'échec
69     reports:
70       junit: build/junit.xml
```

Listing 1. Jobs `build` et `test` (`.gitlab-ci.yml`, lignes 47 à 70).

3.2 Preuves d'exécution

Lancé en root sur une machine à 2 cœurs, `mpirun` refuse deux fois de démarrer avant de réussir avec les deux options demandées par le sujet (listing 2). CTest, configuré avec ces options, passe ses deux tests (listing 3). Le même pipeline, exécuté par l'émulateur avec des artefacts isolés entre jobs, donne `PASS build` et `PASS test` (variante A du tableau 4).

```
$ mpirun -np 4 build/tests/test_mpi 2>&1 | head -3
--------------------------------------------------------------------------
mpirun has detected an attempt to run as root.

[code retour : 1]
$ mpirun --allow-run-as-root -np 4 build/tests/test_mpi 2>&1 | head -3
--------------------------------------------------------------------------
There are not enough slots available in the system to satisfy the 4
slots that were requested by the application:
[code retour : 1]
$ nproc; mpirun --allow-run-as-root --oversubscribe -np 4 build/tests/test_mpi
2
  rang 0 sur vm
  rang 1 sur vm
  rang 2 sur vm
  rang 3 sur vm
MPI test passed with 4 ranks on 1 host(s)
[code retour : 0]
```

Listing 2. `mpirun` en root sur 2 cœurs : refus, refus, puis succès (`logs/extraits/m1-mpirun.txt`).

```
$ cd build && ctest --output-on-failure -j4
Test project /home/catmull/mpiflow/build
    Start 1: mpi_bcast_np1
1/2 Test #1: mpi_bcast_np1 ....................   Passed    0.38 sec
    Start 2: mpi_bcast_np4
```

```
2/2 Test #2: mpi_bcast_np4 ....................   Passed    0.55 sec

100% tests passed, 0 tests failed out of 2

Total Test time (real) =   0.93 sec
[code retour : 0]
```

Listing 3. CTest avec les options du CI (`logs/m1-ctest.txt`).

[À compléter : exécution de `scripts/gitlab_evidence.py` : le log du pipeline de branche sur GitLab.com s'insérera ici]

3.3 Questions de cadrage

3.3.1 Deux jobs d'un même stage échouent : le pipeline continue-t-il ?

Nous avons construit un pipeline de trois builds parallèles, dont deux échouent, suivis d'un stage de test (listing 4). On voit que `build_intel` échoue en premier sans interrompre `build_gcc`, qui échoue 2 s plus tard : les jobs d'un même stage sont indépendants et vont tous au bout. On voit aussi que le job `test` n'apparaît pas : il n'a jamais été lancé, tandis que `cleanup`, déclaré `when: on_failure`, s'est exécuté. La documentation le confirme : les jobs d'un stage ne démarrent qu'après la *réussite* des jobs du stage précédent. Il existe trois exceptions : `allow_failure: true` rend un échec non bloquant, les jobs `when: on_failure` ou `always` servent au nettoyage et aux notifications, et un job relié par `needs:` à des jobs qui ont réussi ne dépend pas des autres. À l'inverse, `workflow:auto_cancel:on_job_failure: all` annule tout au premier échec.

```
$ gitlab-ci-local --force-shell-executor
build_intel > intel échoue aussi, sans être interrompu par gcc
build_intel finished in 22 ms  FAIL 1
build_nvhpc > nvhpc réussit
build_nvhpc finished in 24 ms
build_gcc   > gcc échoue (après 2 s)
build_gcc   finished in 2.03 s  FAIL 1
cleanup     > nettoyage / notification
cleanup     finished in 14 ms
 PASS  build_nvhpc
 PASS  cleanup
 FAIL  build_gcc
 FAIL  build_intel
```

Listing 4. Deux échecs dans le stage `build` (`logs/m1-stage-fail.txt`).

Ce comportement est souhaitable pour trois raisons. Il fait échouer tôt : tester ou empaqueter un binaire qui n'a pas compilé n'a pas de sens, et en HPC l'étape suivante coûte des heures de file d'attente et de quota. Il empêche de publier un artefact non validé : c'est aussi pourquoi notre job `package` déclare `needs: [build, test]`, car `needs: [build]` seul contournerait l'ordre des stages et publierait même si les tests échouent. Enfin, laisser les jobs parallèles aller au bout donne toutes les causes d'échec (ici gcc et intel) dans un seul pipeline.

3.3.2 Quelle différence entre mpirun dans le conteneur CI et une exécution sur cluster ?

Le tableau 3 résume les différences ; nos expériences les rendent concrètes. En CI, il a fallu `--allow-run-as-root` (le conteneur tourne en root) et `--oversubscribe` (4 rangs pour 2 cœurs), et le test affiche `4 ranks on 1 host(s)` : aucune communication n'a traversé un réseau et les temps mesurés n'ont aucun sens. Sur le mini-cluster, *le même binaire* a échoué de deux façons invisibles en CI : lancé par `srun` sans PMIx, Open MPI refuse de démarrer, et réparti sur deux nœuds, un rang a planté pendant que les autres restaient bloqués jusqu'au

Tableau 3. Exécution MPI dans le conteneur CI et sur le cluster.

	Conteneur CI (runner partagé)	Cluster (Slurm)
Placement	1 machine de 2 à 4 vCPU : 4 rangs sur 1 hôte, en surréservation	rangs répartis sur plusieurs nœuds alloués
Lancement	`mpirun` lance lui-même les processus, sans allocation	`srun` dans une allocation Slurm, via PMIx
Communications	mémoire partagée uniquement	réseau entre nœuds, mémoire partagée dans un nœud
Environnement	Open MPI 4.1 de Debian, utilisateur root	MPI du cluster (Spack ou modules), utilisateur sans privilèges
Ce que ça valide	la logique du code sur peu de rangs, à chaque push	l'intégration du code, du MPI, du lanceur et du réseau ; les performances

wall-time (section 5.2). Enfin, le binaire produit dans le conteneur est lié à l'Open MPI de l'image Debian, pas à celui du cluster : les jobs HPC recompilent sur le nœud de connexion. On en déduit que le CI joue le rôle d'un test unitaire du code MPI, et le cluster celui d'un test d'intégration du couple code et environnement.

3.3.3 Pourquoi le test échoue-t-il si l'on oublie needs : alors que le build a réussi ?

L'expérience nous a obligés à nuancer la question : supprimer `needs:` *ne fait pas* échouer le test sur GitLab (tableau 4, variante B). Chaque job démarre dans un environnement neuf (nouveau conteneur, nouveau clone) où `build/` n'existe pas, mais par défaut GitLab télécharge les artefacts de tous les jobs des stages précédents. Le test n'échoue que s'il ne reçoit pas l'artefact : avec `needs: []`, qui le fait démarrer sans attendre (variante C, listing 5), ou s'il est placé dans le même stage que le build (variante D). L'échec « needs oublié » est en revanche systématique sur GitHub Actions, où des jobs sans `needs:` tournent en parallèle et ne partagent aucun fichier.

Tableau 4. Effet de `needs` sur le job `test` (`logs/m1-needs.txt`).

Variante du job `test`	Comportement observé	Résultat
A. `needs: [build]`	artefact importé, `MPI test passed`	réussi
B. `needs:` supprimé	artefact importé quand même (stage précédent)	réussi
C. `needs: []`	démarre sans attendre : `cd: build: No such file or directory`	échec
D. `needs:` supprimé, même stage que `build`	jobs parallèles, pas d'artefact : même erreur	échec

```
### C. needs: [] (démarre sans attendre, sans artefact)
    test:
      stage: test
      extends: [.apt_mpi, .every_pipeline]
      needs: []
$ gitlab-ci-local --force-shell-executor --shell-isolation build test
test  > script: line 7: cd: build: No such file or directory
test  finished in 145 ms  FAIL 1
 PASS  build
 FAIL  test
```

Listing 5. Variante C : `needs: []` (`logs/m1-needs.txt`).

Sur GitLab, `needs:` sert donc à autre chose : le test démarre dès la fin de `build` sans attendre le reste du stage, et seuls les artefacts listés sont téléchargés (sinon les `build/` de plusieurs compilateurs arriveraient tous au même chemin). L'expérience a révélé un second piège : `ctest` indique `Test project .../builds/build/build`, c'est-à-dire le répertoire du job `build`.

Un arbre de build CMake contient des chemins absolus et n'est pas relocalisable ; cela fonctionne sur GitLab.com parce que tous les jobs tournent au même chemin (`/builds/<groupe>/<projet>`), mais nous l'avons payé avec `cpack` en mission 2.

3.4 Erreurs rencontrées et résolution

Tableau 5. Erreurs de la mission 1.

Symptôme	Cause	Résolution
`mpirun has detected an attempt to run as root`	le conteneur CI s'exécute en root	`--allow-run-as-root`
`not enough slots available`	4 rangs demandés, 2 cœurs	`--oversubscribe`, passé à CTest par `MPIEXEC_PREFLAGS`
Le test des diapositives passe malgré un bug volontaire (listing 6)	`Release` ajoute `-DNDEBUG`, qui supprime `assert()`	vérifications explicites et code retour ; vérifier qu'un test *peut* échouer
Jobs en échec `FAIL 100` dans l'émulateur	deux `apt-get` parallèles sur la même machine (exécuteur *shell*)	dépendances préinstallées ; résultats écartés
Non observé (pas de Docker en local)	Open MPI en conteneur Docker : copie mémoire CMA impossible	`OMPI_MCA_btl_vader_single_copy_mechanism=none`, posé préventivement

```
$ mpicc -O2 -o dbg slide_test_mpi_bug.c && mpirun --allow-run-as-root --oversubscribe --tag-
    ↪ output -np 2 ./dbg 2>&1 | grep -E 'Assertion|passed|exited'
[1,1]<stderr>:dbg: slide_test_mpi_bug.c:12: main: Assertion `data == 42' failed.
[1,1]<stderr>:[vm:02657] [1,1]<stderr>:[ 0] [1,0]<stderr>:dbg: slide_test_mpi_bug.c:12: main:
    ↪ Assertion `data == 42' failed.
mpirun noticed that process rank 1 with PID 0 on node vm exited on signal 6 (Aborted).
[code retour : 134]
$ mpicc -O3 -DNDEBUG -o rel slide_test_mpi_bug.c && mpirun --allow-run-as-root --oversubscribe
    ↪  -np 2 ./rel
MPI test passed with 2 ranks
[code retour : 0]
```

Listing 6. Le test des diapositives avec un bug volontaire : détecté sans `-DNDEBUG`, invisible en `Release` (`logs/m1-ndebug.txt`).

3.5 Bilan

Ce qui fonctionne : build CMake, CTest à 1 et 4 rangs, pipeline validé par le schéma GitLab et exécuté de bout en bout par l'émulateur.

Ce qui ne fonctionne pas : le comportement des runners de GitLab.com reste à relever (pas de Docker sur le poste de reproduction).

À refaire autrement : écrire d'abord un test qui échoue avant de lui faire confiance ; c'est ce qui a révélé le piège de `NDEBUG`.

4 Mission 2 : déclencheurs, règles et secrets

4.1 Réalisation

Trois jobs s'ajoutent au pipeline (figure 1). Le job `lint` lance `cppcheck` avec `--error-exitcode=1`, uniquement dans les pipelines de merge request. Le job `package` ne s'exécute que sur les tags `vX.Y.Z` : il archive le binaire, le README et un fichier `BUILDINFO.txt` (projet, tag, commit, URL du pipeline, date), puis calcule le SHA-256 de l'archive. Le job `registry_`

`login` lit la variable masquée et protégée `FAKE_REGISTRY_TOKEN` et la transmet par l'entrée standard à `scripts/fake_docker_login.sh`, qui reproduit le contrat de `docker login --password-stdin` sans démon Docker. Un bloc `workflow:rules` choisit le type de pipeline (listing 7) et les jobs sans condition reçoivent `rules: - when: on_success`.

```
15 # Pipelines de MR quand une MR est ouverte, pipelines de branche sinon : pas de
16 # doublons (un job sans rules vaut except: merge_requests, cf. doc « job rules »).
17 workflow:
18   rules:
19     - if: $CI_PIPELINE_SOURCE == "merge_request_event"
20     - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
21       when: never
22     - if: $CI_COMMIT_BRANCH            # push sur une branche, pipelines planifiés
23     - if: $CI_COMMIT_TAG               # releases
```

Listing 7. Choix du type de pipeline (`.gitlab-ci.yml`, lignes 15 à 23).

4.2 Preuves d'exécution

L'émulateur évalue les règles pour chaque déclencheur (tableau 6) : `lint` n'existe que dans le pipeline de MR, `package` que pour un tag `vX.Y.Z` (pas pour `seance-inversee`, listing 8), et les jobs Slurm que si un runner HPC est déclaré. Sur une branche contenant une erreur volontaire, `cppcheck` échoue avec le code 1 (listing 9). Le pipeline du tag `v0.1.0` produit une archive dont le contenu et la somme de contrôle se vérifient (listing 10).

Tableau 6. Jobs créés selon le déclencheur (`logs/m2-rules.txt`).

Pipeline	`lint`	`build`, `test`	`package`	`registry_login`	`test_slurm`⋆
push sur une branche	–	oui	–	oui	–
merge request	oui	oui	–	oui	–
tag `v1.2.0`	–	oui	oui	oui	–
tag `seance-inversee`	–	oui	–	oui	–
planifié sur `main`, `HPC_ENABLED=true`	–	oui	–	oui	oui
push sur `main`, `HPC_ENABLED=true`	–	oui	–	oui	manuel

```
$ for t in v1.2.0 v0.1.0 v10.20.30 v1.2 1.2.0 v1.2.0-rc1 seance-inversee; do printf '%-16s '
    ↪ $t; [[ $t =~ ^v[0-9]+\.[0-9]+\.[0-9]+$ ]] && echo 'package: OUI' || echo 'package: non';
    ↪  done
v1.2.0           package: OUI
v0.1.0           package: OUI
v10.20.30        package: OUI
v1.2             package: non
1.2.0            package: non
v1.2.0-rc1       package: non
seance-inversee  package: non
[code retour : 0]
```

Listing 8. Quels tags déclenchent `package` ? (`logs/m2-regex.txt`).

```
$ cppcheck --enable=warning,style,performance,portability --std=c11 --error-exitcode=1 --
    ↪ inline-suppr --suppress=missingIncludeSystem tests/   # branche avec erreurs
    ↪ volontaires
Checking tests/test_mpi.c ...
tests/test_mpi.c:28:36: error: Array 'tab[4]' accessed at index 4, which is out of bounds. [
    ↪ arrayIndexOutOfBounds]
    int errors = 0; int tab[4]; tab[4] = rank;
                                   ^
tests/test_mpi.c:28:40: style: Variable 'tab[4]' is assigned a value that is never used. [
    ↪ unreadVariable]
```

```
    int errors = 0; int tab[4]; tab[4] = rank;
                                        ^
[code retour : 1]
```

Listing 9. Lint sur une branche contenant une erreur volontaire (`logs/m2-lint.txt`).

```
$ gitlab-ci-local --force-shell-executor --shell-isolation --variable CI_COMMIT_TAG=v0.1.0
    ↪ build test package
[... mkdir, cp, printf, tar : voir le job package ci-dessus ...]
package $ cd dist && sha256sum "$PKG.tar.gz" | tee checksums.txt
package > 9ed16eb3059d01e2e572a22c96e11b27ae19f81683a9e962f71715b58063c5a3  mpiflow-v0.1.0-
    ↪ linux-x86_64.tar.gz
package finished in 336 ms
 PASS  build
 PASS  test
 PASS  package
$ tar tzf dist/mpiflow-v0.1.0-linux-x86_64.tar.gz
mpiflow-v0.1.0-linux-x86_64/
mpiflow-v0.1.0-linux-x86_64/README.md
mpiflow-v0.1.0-linux-x86_64/BUILDINFO.txt
mpiflow-v0.1.0-linux-x86_64/bin/
mpiflow-v0.1.0-linux-x86_64/bin/test_mpi
$ tar xzf dist/mpiflow-v0.1.0-linux-x86_64.tar.gz -O mpiflow-v0.1.0-linux-x86_64/BUILDINFO.txt
projet=chps/mpiflow
tag=v0.1.0
commit=def6b052a2214d2edf4827d7128ae1b3c43e3995
pipeline=https://gitlab.com/chps/mpiflow/pipelines/0
date=2026-10-24T14:04:15Z
$ cd dist && sha256sum -c checksums.txt
mpiflow-v0.1.0-linux-x86_64.tar.gz: OK
```

Listing 10. Pipeline du tag `v0.1.0` : archive, contenu et somme de contrôle (`logs/extraits/m2-package.txt`).

Pour les secrets, le listing 11 montre qu'un utilisateur sans privilèges lit en clair, avec `ps`, le mot de passe passé en argument à un processus de root, alors que `--password-stdin` ne laisse rien dans la ligne de commande. L'émulateur n'applique ni le masquage ni la protection des variables : ces deux mécanismes ne se vérifient que sur GitLab.com (figure **??**).

```
$ python3 -c 'import time; time.sleep(3)' docker login -u ci-user -p mypassword123 & sleep 1;
    ↪ runuser -u etudiant -- ps -eo user:8,args | grep -- '-p mypassword' | grep -v grep
root     python3 -c import time; time.sleep(3) docker login -u ci-user -p mypassword123
[code retour : 0]
$ export FAKE_REGISTRY_TOKEN=FAUX-jeton-ci-0123456789; echo "$FAKE_REGISTRY_TOKEN" | ./scripts/
    ↪ fake_docker_login.sh -u ci-user --password-stdin registry.example.org
Login Succeeded (simulation) : ci-user@registry.example.org, secret de 24 caractères reçu via
    ↪ stdin
[code retour : 0]
$ echo '' | ./scripts/fake_docker_login.sh -u ci-user --password-stdin registry.example.org
    ↪  # variable protégée absente
Erreur : aucun secret reçu sur stdin (variable vide ?)
[code retour : 1]
$ ./scripts/fake_docker_login.sh -u ci-user -p mypassword123
Refusé : -p expose le secret dans la ligne de commande (ps, /proc, historique).
[code retour : 2]
```

Listing 11. Mot de passe en argument (`-p`) contre entrée standard (`logs/m2-secret.txt`).

[À compléter : exécution de `scripts/gitlab_evidence.py` : masquage sur `main`, merge request avec lint en échec et pipeline du tag `v0.1.0` s'inséreront ici]

4.3 Questions de cadrage

4.3.1 rules : ou only :/except : : lequel écrire, et où retrouver l'autre ?

Il faut écrire `rules:`. La documentation de GitLab classe désormais `only` et `except` parmi les mots-clés dépréciés et renvoie vers `rules`. On les retrouve dans les dépôts et gabarits anciens, comme le pipeline « hérité » des diapositives : il faut savoir les lire et les traduire (`only: merge_requests` devient `if: $CI_PIPELINE_SOURCE == "merge_request_event"`). Les deux ne se combinent pas dans un même job : le validateur de GitLab rejette la configuration (`key may not be used with rules`), alors que l'émulateur, plus laxiste, l'accepte.

`rules:` est plus expressif : conditions sur des variables, `changes`, `exists`, première règle vraie gagnante, `when: manual` ou `never` et `allow_failure` par règle ; nous nous en servons pour rendre les jobs HPC manuels sur `main` et automatiques la nuit. Mélanger les deux styles reste un piège, car un job sans règle vaut implicitement `except: merge_requests`. Avec un `lint` limité aux MR et des jobs `build` et `test` sans règle, chaque push sur une branche ayant une MR ouverte crée deux pipelines : un pipeline de branche (build et test) et un pipeline de MR (lint seul). D'où le `workflow:rules` du listing 7.

4.3.2 Que devient une variable Protected sur une branche non protégée ?

Elle n'est pas injectée : le job démarre avec une variable vide, sans erreur. Notre job affiche alors `FAKE_REGISTRY_TOKEN vide`, et `fake_docker_login.sh` refuse un secret vide (code retour 1) ; un vrai job de déploiement doit échouer ainsi, explicitement, plutôt que de continuer sans identifiant.

Le risque couvert est l'exfiltration par modification du pipeline. Toute personne qui peut pousser une branche peut aussi modifier le `.gitlab-ci.yml` de cette branche : un simple `echo $TOKEN | base64` ou un `curl` vers un serveur externe contourne le masquage, qui ne fait que remplacer la valeur exacte dans le log (la documentation précise d'ailleurs que le masquage n'est pas une garantie). La protection réserve les secrets aux branches et tags protégés, où seuls les mainteneurs poussent ou fusionnent, donc à du code relu en merge request. Pour notre pipeline, la clé SSH du cluster et le futur jeton de buildcache sont donc protégés, et le runner `hpc-login` doit lui aussi être déclaré protégé : sinon, n'importe quelle branche pourrait exécuter du code arbitraire sur le nœud de connexion.

4.3.3 Pourquoi ne jamais écrire un secret dans le .gitlab-ci.yml, même temporairement ?

Parce que « temporairement » n'existe pas dans Git. Un secret commité reste dans l'historique même après le commit qui le supprime, ainsi que dans tous les clones, forks, diffs de MR et caches de GitLab ; l'effacer exige de réécrire l'historique, et le secret doit de toute façon être considéré comme compromis et révoqué. Le fichier est en outre lisible par quiconque a accès au dépôt (tout le monde si le projet est public, et des robots parcourent les forges à la recherche de secrets). Enfin, une valeur écrite en clair n'est pas masquée : le masquage ne s'applique qu'aux variables déclarées dans *Settings > CI/CD*, et GitLab recopie chaque ligne de script dans le log du job. Passée en argument (`-p`), elle apparaît en plus dans `ps` pour les autres utilisateurs de la machine (listing 11).

Les variables de l'interface ont leurs propres contraintes : une valeur masquée doit tenir sur une ligne, sans espace, et compter au moins 8 caractères (une fausse valeur trop courte comme `1234` ne peut pas être masquée). Une variable de type *File* contient le chemin d'un fichier temporaire, ce qui convient à une clé SSH que `env` n'affichera pas.

Tableau 7. Erreurs de la mission 2.

Symptôme	Cause	Résolution
Deux pipelines par push sur une branche avec MR ouverte (analyse des règles)	un job sans `rules` vaut `except: merge_requests`	`workflow:rules` et `rules: - when: on_success`
Archive `cpack` sans `BUILDINFO.txt` (`logs/m2-package-cpack.txt`)	le chemin du build est figé dans le cache CMake : il pointe vers le répertoire du job `build`	archive construite avec `tar`, vérifiée par `tar tzf` et `sha256sum -c`
`package` pouvait publier un binaire non testé	`needs: [build]` contourne l'ordre des stages	`needs: [build, test]`, sans les artefacts de `test`
Masquage invisible en local	l'émulateur n'implémente pas le masquage	vérification sur GitLab.com

4.4 Erreurs rencontrées et résolution

4.5 Bilan

Ce qui fonctionne : règles validées pour six types de pipelines, lint qui échoue sur une vraie erreur, archive vérifiable (contenu, `BUILDINFO`, SHA-256), secret transmis par l'entrée standard.

Ce qui ne fonctionne pas : masquage et protection non observables hors de GitLab.com ; le binaire de l'archive dépend de l'Open MPI de Debian et n'est pas autonome.

À refaire autrement : poser `workflow:rules` dès le premier job conditionnel, et produire les archives de version à partir de l'environnement Spack plutôt que d'une image Debian.

5 Mission 3 : du CI vers Slurm

5.1 Réalisation

Le job Slurm `slurm/job_ci.sh` (annexe B) demande 2 nœuds de 2 tâches (nœuds `e2-standard-2` à 2 vCPU), 5 minutes et la partition `debug`. Il charge l'environnement dans l'ordre Lmod, environnement Spack du dépôt, puis système, et affiche la source retenue ; sur le cluster du Cours 5, sans Lmod, c'est Spack. Il lance `srun --mpi=pmix --kill-on-bad-exit=1` avec `EXPECTED_RANKS=$SLURM_NTASKS` et se termine par `SUCCESS` ou `FAILED`.

Le script `slurm/submit_and_poll.sh` (annexe B) soumet le job par `sbatch --parsable`, enregistre son identifiant dans `.slurm_jobid`, puis interroge `sacct -X` (repli sur `squeue` tant que la base n'a pas l'entrée) et affiche l'état, la raison d'attente et le temps écoulé. Il reconnaît tous les états terminaux, y compris ceux qu'oublie la boucle des diapositives (`OUT_OF_MEMORY`, `NODE_FAIL`, `PREEMPTED`...), archive `sacct` et la sortie du job, et rend 0 (succès), 1 (échec du job) ou 75 (échec d'infrastructure).

Côté GitLab (listing 12), un gabarit `.hpc` porte le tag `hpc-login`, attend le job `test` sans récupérer son binaire, recompile avec le MPI du cluster et annule le job Slurm dans `after_script`. Deux jobs l'utilisent : `test_slurm` (suivi, `timeout`, `retry`) et `test_slurm_wait` (`sbatch --wait`).

```
    SLURM_OPTS: ""                      # ex. "--partition=compute" (vérifier avec sinfo)
    # script coupé à 40 min pour laisser à after_script le temps d'annuler le job
    RUNNER_SCRIPT_TIMEOUT: 40m
    RUNNER_AFTER_SCRIPT_TIMEOUT: 3m
  after_script:
    # exécuté aussi si le job est annulé ou coupé : pas de job Slurm orphelin
    - |
      if [ -s .slurm_jobid ] && squeue -h -j "$(cat .slurm_jobid)" 2>/dev/null | grep -q .;
    ↪ then
        scancel "$(cat .slurm_jobid)" && echo "Job Slurm $(cat .slurm_jobid) annulé"
```

```
      fi
  artifacts:
test_slurm:
  extends: .hpc
  timeout: 45m                          # > wall-time Slurm (5 min) + attente en file
  retry:
    max: 1
    when: [runner_system_failure, runner_interrupted]
    exit_codes: 75                      # NODE_FAIL / BOOT_FAIL / PREEMPTED (submit_and_poll.sh)
    ↪
  script:
    - ./slurm/submit_and_poll.sh slurm/job_ci.sh $SLURM_OPTS

test_slurm_wait:
  extends: .hpc
  timeout: 45m
  script:
    - sbatch --wait --parsable $SLURM_OPTS slurm/job_ci.sh > .slurm_jobid
    - cat "ci-$(cat .slurm_jobid).out"
```

Listing 12. Délais, annulation et jobs Slurm (`.gitlab-ci.yml`, extrait des lignes 140 à 185).

Pour atteindre le cluster, le sujet propose un runner installé sur le nœud de connexion ou une connexion SSH depuis un runner partagé (tableau 8). Nous retenons le runner sur le nœud de connexion (figure 2) : il n'ouvre aucune connexion entrante, partage `/home` avec les nœuds de calcul et ne consomme pas de minutes de runner partagé pendant l'attente en file. La variante SSH (`test_slurm_ssh`, `slurm/ci_remote.sh`) est écrite mais ne s'active que si les variables SSH existent. Le script `scripts/gcp_evidence.sh` enregistre le runner (exécuteur shell) quand on lui fournit le jeton créé dans l'interface de GitLab, où sont fixés le tag `hpc-login` et la protection.

Tableau 8. Deux façons d'atteindre le cluster depuis GitLab.

	Runner sur le nœud de connexion	SSH depuis un runner partagé
Réseau	connexion sortante uniquement (le runner interroge GitLab en HTTPS)	IP publique et port 22 ouverts ; les IP des runners partagés ne sont pas fixes
Secrets	aucun secret de connexion dans GitLab	clé privée en variable *File* et *Protected*, `known_hosts`
Fichiers	build sur `/home` (NFS), visible des nœuds	`rsync` aller-retour des sources et des logs
Attente en file	ne coûte rien côté GitLab	minutes de runner partagé consommées
Précautions	runner protégé, utilisateur dédié, aucun calcul sur le nœud de connexion	coupure SSH = job perdu ; `after_script` dans un shell séparé

5.2 Preuves d'exécution

Toutes les exécutions de cette section ont lieu sur le mini-cluster, soumises par un utilisateur sans privilèges ; la figure 3 les replace sur une même échelle de temps. Le cas nominal est donné par le listing 13 : 4 rangs sur 2 nœuds, `ExitCode 0:0`, ligne `SUCCESS`. Une première version du script interrogeait `sacct` dès l'état terminal ; la comptabilité étant asynchrone, le nom du job et la liste des nœuds n'étaient pas encore à jour (listing 14). Le script attend désormais quelques secondes avant d'archiver.

```
$ runuser -l etudiant -c 'cd mpiflow && export OMPI_MCA_btl=self,tcp && POLL_INTERVAL=3 ./
    ↪ slurm/submit_and_poll.sh slurm/job_ci.sh'
Job Slurm 20 soumis (2026-10-24T14:28:22+00:00)
[14:28:22] +    0s  job 20  PENDING    raison=None
[14:28:25] +    3s  job 20  COMPLETED
---- sacct ----
JobID               JobName      State ExitCode    Elapsed   NNodes          NodeList
------------ --------------- ---------- -------- ---------- -------- -----------------
20               ci-mpiflow  COMPLETED      0:0   00:00:01        2        node[1-2]
```

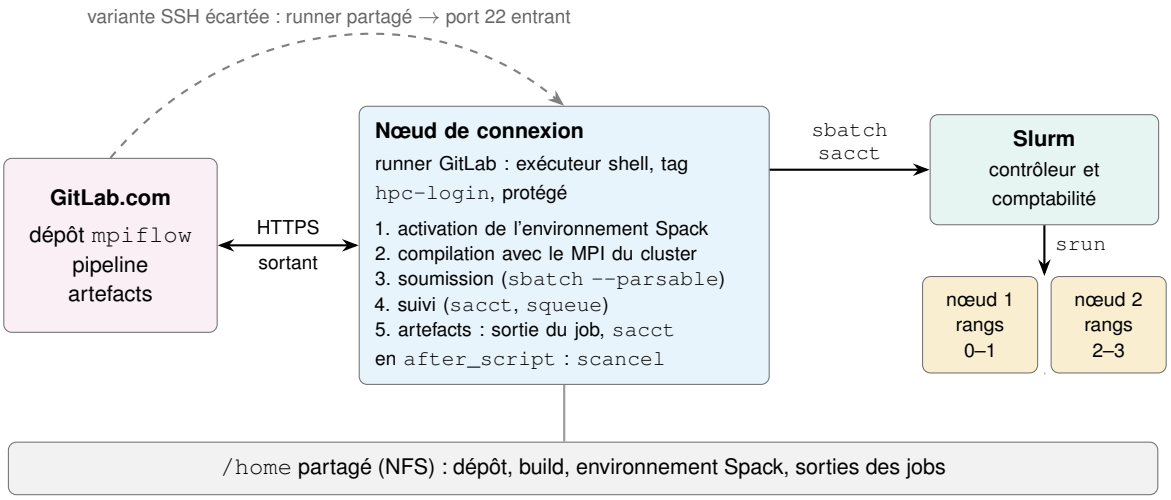


Figure 2. Architecture retenue pour la mission 3. Le runner orchestre depuis le nœud de connexion et ne calcule jamais : le calcul a lieu sur les nœuds alloués par Slurm. Le `/home` partagé rend le binaire compilé et les sorties du job visibles des deux côtés. En tirets, la variante SSH écartée.

```
20.batch              batch  COMPLETED      0:0   00:00:01        1            node1
20.0               test_mpi  COMPLETED      0:0   00:00:00        2        node[1-2]
---- sortie du job ----
== Job 20 : 2 nœud(s) [node[1-2]], 4 tâches
== Soumis depuis /home/etudiant/mpiflow (commit hors CI)
== Environnement : système (PATH par défaut)
== mpicc : /usr/bin/mpicc
  rang 0 sur vm
  rang 1 sur vm
  rang 2 sur vm
  rang 3 sur vm
MPI test passed with 4 ranks on 1 host(s)
SUCCESS: MPI test passed (4 rangs sur 2 nœuds)
Verdict : succès
[code retour : 0]
```

Listing 13. Soumission et suivi : succès sur 2 nœuds (`logs/m3-poll-4-success.txt`).

```
$ sacct -j 19 -o JobID,JobName%14,State,ExitCode,Elapsed,NNodes,NodeList%16   # juste après
    ↪ COMPLETED
JobID                JobName      State ExitCode    Elapsed   NNodes         NodeList
------------ -------------- ---------- -------- ---------- -------- ----------------
19                allocation  COMPLETED      0:0   00:00:01        2            node1
19.batch              batch  COMPLETED      0:0   00:00:01        1            node1
19.0                test_mpi  COMPLETED      0:0   00:00:00        2        node[1-2]
$ sleep 3; sacct -j 19 -X -o JobID,JobName%14,State,NodeList%12
JobID                JobName      State     NodeList
------------ -------------- ---------- ------------
19                ci-mpiflow  COMPLETED    node[1-2]
```

Listing 14. `sacct` interrogé trop tôt, puis 3 s plus tard (`logs/m3-sacct-race.txt`).

Un rang plante, les autres attendent. Notre première soumission sur deux nœuds a mis plus de cinq minutes à échouer (listing 15) : deux rangs ont planté dans le transport mémoire partagée `vader`, les deux autres sont restés bloqués dans la collective jusqu'à ce que Slurm tue le job au wall-time. Avec `srun --kill-on-bad-exit=1`, le même échec est détecté en une seconde (listing 16). Le plantage lui-même vient de l'émulation : les deux nœuds Slurm sont la même machine, et Open MPI y mélange ses segments de mémoire partagée. Sur un seul nœud, le test passe, et sur deux nœuds avec `OMPI_MCA_btl=self,tcp` aussi (listing 13) ; ce réglage ne concerne que le mini-cluster.

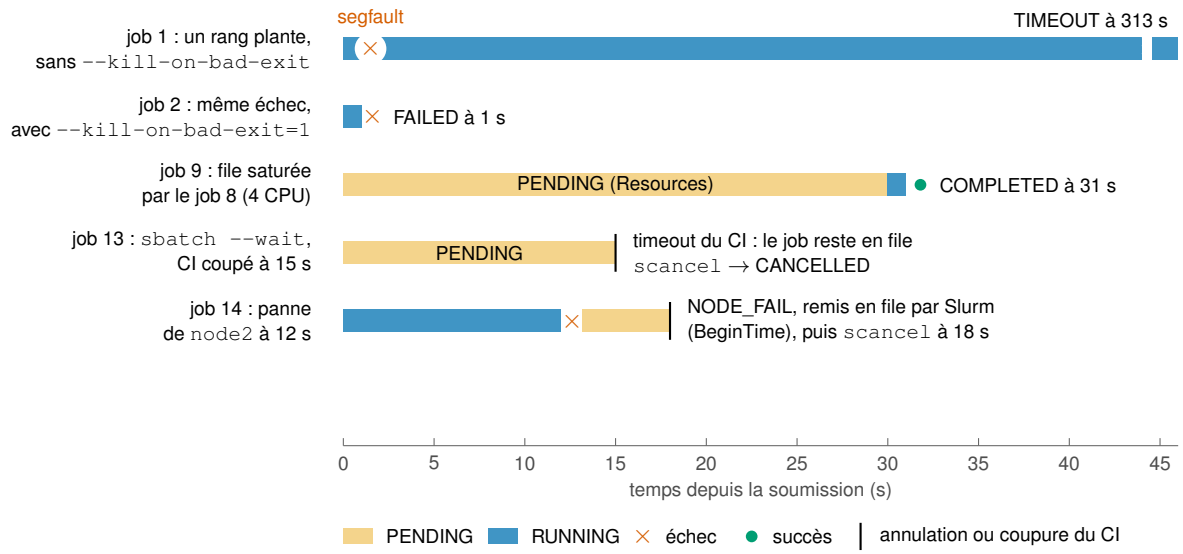


Figure 3. Chronologie des jobs Slurm des expériences de la mission 3, d'après les heures de soumission, de début et de fin enregistrées par `slurmdbd` (`logs/m3-chronologie.txt`) et le journal de `slurmd` pour l'instant du plantage. L'axe du job 1 est interrompu : ses rangs restants sont restés bloqués jusqu'au wall-time de 5 minutes.

```
$ runuser -l etudiant -c 'cd mpiflow && POLL_INTERVAL=5 ./slurm/submit_and_poll.sh slurm/
    ↪ job_ci.sh'
Job Slurm 1 soumis (2026-10-24T13:45:36+00:00)
[13:45:36] +    0s  job 1  PENDING    raison=None
   début estimé par Slurm : N/A
[13:45:41] +    5s  job 1  RUNNING    raison=None
[13:45:46] +   10s  job 1  RUNNING    raison=None
[... 57 lignes RUNNING identiques, une toutes les 5 s ...]
[13:50:44] +  308s  job 1  RUNNING    raison=None
---- sacct ----
JobID               JobName      State ExitCode    Elapsed   NNodes         NodeList
------------ --------------- ---------- -------- ---------- -------- ----------------
1                ci-mpiflow    TIMEOUT      0:0   00:05:13        2        node[1-2]
1.batch               batch  CANCELLED     0:15   00:05:13        1            node1
1.0                test_mpi  CANCELLED     0:15   00:05:12        2        node[1-2]
---- sortie du job ----
== Job 1 : 2 nœud(s) [node[1-2]], 4 tâches
== Soumis depuis /home/etudiant/mpiflow (commit hors CI)
== Environnement : système (PATH par défaut)
== mpicc : /usr/bin/mpicc
[vm:04044] *** Process received signal ***
[vm:04044] Signal: Segmentation fault (11)
[vm:04044] [ 1] /usr/lib/x86_64-linux-gnu/openmpi/lib/openmpi3/mca_btl_vader.so(
    ↪ mca_btl_vader_poll_handle_frag+0x14c)[0x7f3bd01f4dac]
[... trace complète dans logs/m3-poll-1-timeout.txt ...]
srun: error: node1: tasks 0-1: Segmentation fault
srun: Job step aborted: Waiting up to 32 seconds for job step to finish.
slurmstepd-node1: error: *** JOB 1 ON node1 CANCELLED AT 2026-10-24T13:50:49 DUE TO TIME LIMIT
    ↪  ***
srun: got SIGCONT
srun: forcing job termination
srun: error: node2: tasks 2-3: Terminated
Verdict : échec du JOB (TIMEOUT, ExitCode=0:0)
[code retour : 1]
```

Listing 15. Échec lent : TIMEOUT Slurm après 313 s (`logs/extraits/m3-poll-1-timeout.txt`).

```
$ runuser -l etudiant -c 'cd mpiflow && POLL_INTERVAL=5 ./slurm/submit_and_poll.sh slurm/
    ↪ job_ci.sh'
Job Slurm 2 soumis (2026-10-24T13:51:01+00:00)
[13:51:01] +    0s  job 2  PENDING    raison=None
```

```
   début estimé par Slurm : N/A
[13:51:06] +    5s  job 2  FAILED
---- sacct ----
JobID               JobName      State ExitCode    Elapsed   NNodes         NodeList
------------ -------------- ---------- -------- ---------- -------- ----------------
2                ci-mpiflow     FAILED     15:0   00:00:01        2        node[1-2]
2.batch               batch     FAILED     15:0   00:00:01        1            node1
2.0                 test_mpi  CANCELLED     0:15   00:00:00        2        node[1-2]
---- sortie du job ----
== Job 2 : 2 nœud(s) [node[1-2]], 4 tâches
== Soumis depuis /home/etudiant/mpiflow (commit hors CI)
== Environnement : système (PATH par défaut)
== mpicc : /usr/bin/mpicc
[vm:05032] *** Process received signal ***
[vm:05032] Signal: Segmentation fault (11)
[vm:05032] Signal code: Address not mapped (1)
[vm:05032] Failing at address: 0x555b3bd848e0
[vm:05032] [ 0] /lib/x86_64-linux-gnu/libc.so.6(+0x45330)[0x7f0ae9445330]
```

Listing 16. Échec rapide avec `--kill-on-bad-exit=1` (`logs/m3-poll-2-kill.txt`).

File saturée. Un premier job occupe les 4 CPU du cluster ; le job CI reste `PENDING (Resources)` pendant 30 s et le suivi l'affiche à chaque tour (listing 17).

```
$ runuser -l etudiant -c 'sbatch -J occupe-tout --nodes=2 --ntasks=4 --time=00:02:00 --wrap "
    ↪ sleep 30"' && sleep 1 && squeue
Submitted batch job 8
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                 8     debug occupe-t etudiant  R       0:01      2 node[1-2]
[code retour : 0]
$ runuser -l etudiant -c 'cd mpiflow && export OMPI_MCA_btl=self,tcp && POLL_INTERVAL=5 ./
    ↪ slurm/submit_and_poll.sh slurm/job_ci.sh' | sed '/sortie du job/,$d'
Job Slurm 9 soumis (2026-10-24T13:52:30+00:00)
[13:52:30] +    0s  job 9  PENDING    raison=None
[13:52:35] +    5s  job 9  PENDING    raison=Resources
   début estimé par Slurm : N/A
[13:52:40] +   10s  job 9  PENDING    raison=Resources
[13:52:45] +   15s  job 9  PENDING    raison=Resources
[13:52:50] +   20s  job 9  PENDING    raison=Resources
[13:52:55] +   25s  job 9  PENDING    raison=Resources
[13:53:00] +   30s  job 9  PENDING    raison=Resources
[13:53:05] +   35s  job 9  COMPLETED
---- sacct ----
JobID               JobName      State ExitCode    Elapsed   NNodes         NodeList
------------ -------------- ---------- -------- ---------- -------- ----------------
9                ci-mpiflow  COMPLETED      0:0   00:00:01        2        node[1-2]
9.batch               batch  COMPLETED      0:0   00:00:01        1            node1
9.0                 test_mpi  COMPLETED      0:0   00:00:00        2        node[1-2]
[code retour : 0]
```

Listing 17. Job CI en attente derrière un job qui occupe tout le cluster (`logs/m3-pending.txt`).

`sbatch --wait`. Le code retour est bien propagé (0 pour un succès, 1 pour un échec, `logs/extraits/m3-wait-ok-fail.txt`), mais rien ne s'affiche pendant l'attente. Surtout, quand le CI coupe l'attente (un `timeout 15` simule ici le `timeout:` de GitLab), le job Slurm reste en file et s'exécuterait plus tard pour rien (listing 18).

```
$ runuser -l etudiant -c 'sbatch -J occupe-tout --nodes=2 --ntasks=4 --time=00:02:00 --wrap "
    ↪ sleep 60"'
Submitted batch job 12
[code retour : 0]
$ runuser -l etudiant -c 'cd mpiflow && timeout 15 sbatch --wait slurm/job_ci.sh'; echo "(job
    ↪ CI tué par le timeout simulé)"; squeue
Submitted batch job 13
(job CI tué par le timeout simulé)
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                13     debug ci-mpifl etudiant PD       0:00      2 (Resources)
```

```
               12     debug occupe-t etudiant  R       0:15      2 node[1-2]
[code retour : 0]
$ J=$(squeue -h -u etudiant -n ci-mpiflow -o %i); runuser -l etudiant -c "scancel $J"; sleep 2;
    ↪  sacct -X -j $J -o JobID,JobName%12,State
JobID             JobName      State
------------ ------------ ----------
13             ci-mpiflow CANCELLED+
[code retour : 0]
```

Listing 18. `sbatch --wait` coupé par le CI : job orphelin (`logs/m3-wait-orphan.txt`).

Panne de nœud. Nous passons `node2` en `DOWN` pendant un job : le script classe l'échec comme une panne d'infrastructure (code 75). Mais Slurm a déjà remis le job en file (`BeginTime`) : sans `scancel`, il repartirait en plus de la relance éventuelle du CI (listing 19).

```
$ runuser -l etudiant -c 'cd mpiflow && POLL_INTERVAL=4 ./slurm/submit_and_poll.sh job_long.sh
    ↪ '
Job Slurm 14 soumis (2026-10-24T13:53:48+00:00)
[13:53:48] +    0s  job 14  PENDING    raison=None
[13:53:52] +    4s  job 14  RUNNING    raison=None
[13:53:56] +    8s  job 14  RUNNING    raison=None
[13:54:00] +   12s  job 14  NODE_FAIL  raison=BeginTime
---- sacct ----
JobID               JobName      State ExitCode    Elapsed   NNodes         NodeList
------------ ---------- ---------- -------- ---------- -------- ---------------
14                ci-long  NODE_FAIL      0:0   00:00:12        2        node[1-2]
14.batch            batch  CANCELLED     0:15   00:00:12        1            node1
14.0                sleep  CANCELLED     0:15   00:00:11        2        node[1-2]
---- sortie du job ----
srun: error: Node failure on node2
slurmstepd-node1: error: *** STEP 14.0 ON node1 CANCELLED AT 2026-10-24T13:54:00 DUE TO NODE
    ↪ FAILURE, SEE SLURMCTLD LOG FOR DETAILS ***
slurmstepd-node1: error: *** JOB 14 ON node1 CANCELLED AT 2026-10-24T13:54:00 DUE TO NODE
    ↪ FAILURE, SEE SLURMCTLD LOG FOR DETAILS ***
srun: Job step aborted: Waiting up to 32 seconds for job step to finish.
srun: got SIGCONT
srun: forcing job termination
srun: error: node2: task 1: Terminated
srun: error: node1: task 0: Terminated
Verdict : échec d'INFRASTRUCTURE (NODE_FAIL) -> relance possible
[code retour : 75]
$ squeue ; sacct -j 14 -D -X -o JobID,State,Start,End      # juste après le verdict
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                14     debug  ci-long etudiant PD       0:00      2 (BeginTime)
JobID             State               Start                 End
------------ ---------- ------------------- -------------------
14            NODE_FAIL 2026-10-24T13:53:48 2026-10-24T13:54:00
14              PENDING             Unknown             Unknown
$ scancel 14     # ce que fait l'after_script du job GitLab
```

Listing 19. Panne de nœud : verdict d'infrastructure et remise en file par Slurm (`logs/m3-nodefail.txt`).

Base de comptabilité arrêtée. Pendant une session, la base MariaDB du mini-cluster s'est arrêtée, et avec elle `slurmdbd` : `sacct` ne répondait plus. La première version du script a alors affiché `INCONNU` pendant onze minutes sans jamais conclure, exactement le blocage que seul le `timeout:` du CI aurait arrêté. Le script interroge désormais le contrôleur (`scontrol show job`, qui garde les jobs terminés 5 minutes) quand ni `sacct` ni `squeue` ne répondent, et rend le code 75 après six tours sans état. Relancé avec la base toujours arrêtée, il conclut normalement (listing 20).

```
# Base de comptabilité arrêtée pendant le suivi (MariaDB, donc slurmdbd, indisponible) :
$ sacct -j 21 -X -o JobID,State,ExitCode
sacct: error: slurm_persist_conn_open_without_init: failed to open persistent connection to
    ↪ host:vm:6819: Connection refused
```

```
sacct: error: Sending PersistInit msg: Connection refused
sacct: error: Problem talking to the database: Connection refused
# Première version du script : sans état connu, la boucle ne s arrête jamais
$ POLL_INTERVAL=10 ./slurm/submit_and_poll.sh slurm/job_ci.sh --partition=debug --nodes=2 --
    ↪ ntasks=4 --ntasks-per-node=2
Job Slurm 21 soumis (2026-10-24T18:52:20+00:00)
[18:52:20] +    0s  job 21  PENDING    raison=None
[18:52:30] +   10s  job 21  INCONNU
[18:52:40] +   20s  job 21  INCONNU
[...]
[19:03:21] +  661s  job 21  INCONNU
# (interrompu à la main après 11 minutes)
# Script corrigé (repli sur scontrol), base toujours arrêtée :
$ runuser -l etudiant -c 'cd mpiflow && export OMPI_MCA_btl=self,tcp && POLL_INTERVAL=3
    ↪ SACCT_SETTLE=1 ./slurm/submit_and_poll.sh slurm/job_ci.sh' | grep -vE '^  rang|^== (
    ↪ Soumis|Environnement|mpicc)'
Job Slurm 22 soumis (2026-10-24T19:04:15+00:00)
[19:04:15] +    0s  job 22  PENDING    raison=None
[19:04:18] +    3s  job 22  COMPLETED
---- sacct ----
sacct: error: slurm_persist_conn_open_without_init: failed to open persistent connection to
    ↪ host:vm:6819: Connection refused
sacct: error: Sending PersistInit msg: Connection refused
sacct: error: Problem talking to the database: Connection refused
---- sortie du job ----
== Job 22 : 2 nœud(s) [node[1-2]], 4 tâches
MPI test passed with 4 ranks on 1 host(s)
SUCCESS: MPI test passed (4 rangs sur 2 nœuds)
Verdict : succès
[code retour : 0]
```

Listing 20. Suivi avec la base de comptabilité arrêtée, avant puis après correction (`logs/m3-sacct-down.txt`).

srun sans PMI. Sans `--mpi=pmix`, Open MPI détecte le lancement direct par `srun` et refuse de démarrer (`logs/extraits/m3-singleton.txt`). Le contrôle `EXPECTED_RANKS` reste utile avec d'autres implémentations MPI, qui peuvent démarrer en « singletons » de taille 1 et réussir à tort (`logs/m1-singleton.txt`).

Sur le cluster GCP. [À compléter : exécution de `scripts/gcp_evidence.sh` sur le nœud de connexion : la soumission sur deux nœuds réels s'insérera ici]

5.3 Polling ou `sbatch --wait` ?

Tableau 9. Suivi par polling et `sbatch --wait`, d'après nos essais.

	Polling (`submit_and_poll.sh`)	`sbatch --wait`
Taille	script d'une soixantaine de lignes, réutilisable	une à trois lignes
Pendant l'attente	état, raison et temps écoulé à chaque tour	silence total
Code retour	vérifié (`ExitCode` et ligne `SUCCESS`) et classé : code 1, infrastructure 75	propagé tel quel, sans la cause
Après un timeout du CI	identifiant connu, `after_script` annule le job	job orphelin, sauf `--parsable > .slurm_jobid`
Charge	une requête `sacct` par intervalle (30 s)	aucune requête du script

Le tableau 9 compare les deux versions. Nous choisissons le polling pour le pipeline principal et le nightly : la file est imprévisible, et il faut savoir où en était le job et distinguer les échecs. `sbatch --wait` convient aux jobs courts sur une partition de debug peu chargée et aux scripts

qui enchaînent des soumissions (comme la boucle de passage à l'échelle des diapositives), à condition d'écrire l'identifiant du job pour pouvoir l'annuler.

5.4 Questions de cadrage

5.4.1 Pourquoi le job CI doit-il porter un timeout : alors que le job Slurm a un wall-time ?

Parce que les deux horloges ne mesurent pas la même chose. Le wall-time de Slurm ne compte que le temps passé en `RUNNING` ; l'attente en `PENDING` n'est bornée par rien (30 s dans notre expérience, des heures sur un cluster chargé). La durée du job CI vaut attente, plus exécution, plus latence du suivi. Le job CI peut aussi se bloquer indépendamment de Slurm : la boucle des diapositives tourne indéfiniment sur un état qu'elle ne connaît pas (`NODE_FAIL`, `OUT_OF_MEMORY`...) ou si `sacct` ne répond plus, et une connexion SSH peut rester pendante. Pendant ce temps, le job occupe un emplacement de runner, et des minutes facturées dans la variante SSH.

Le `timeout:` doit donc dépasser le wall-time plus une attente raisonnable, sans quoi il tuerait un job légitime : nous avons 5 min de wall-time et 45 min de `timeout`. Cela ne suffit pas : à l'expiration, GitLab arrête le script, et si `after_script` n'a pas le temps de s'exécuter, le job Slurm reste en file. La documentation impose que `RUNNER_SCRIPT_TIMEOUT` et `RUNNER_AFTER_SCRIPT_TIMEOUT` ne dépassent pas ensemble le `timeout` du job, d'où 40 min et 3 min pour 45 min (listing 12). Notre premier essai illustre l'autre sens : un rang planté, les autres bloqués, et seul le wall-time de Slurm a mis fin au gâchis. Les deux garde-fous sont complémentaires.

5.4.2 Le job Slurm reste PENDING 20 minutes : que se passe-t-il côté GitLab ?

Côté GitLab, le job est « running » : le runner exécute la boucle de suivi. Il occupe un emplacement du runner, si bien que les autres pipelines attendent, consomme des minutes de runner partagé dans la variante SSH, et bloque la suite du pipeline (et une MR si le job est requis). Avec `sbatch --wait`, le log reste vide pendant 20 minutes : impossible de distinguer une file pleine d'un blocage. Si l'attente et l'exécution dépassent le `timeout`, le job CI échoue et le job Slurm reste en file s'il n'est pas annulé (figure 3, job 13).

Pour le rendre observable, notre script affiche à chaque tour l'heure, le temps écoulé, l'état et la raison donnée par Slurm (`Resources`, `Priority`, `QOSMaxJobsPerUserLimit`, `ReqNodeNotAvail`...), puis une fois l'heure de démarrage estimée par `squeue --start`, qui reste `N/A` tant que l'ordonnanceur n'a pas calculé de prévision. L'identifiant est conservé, et `sacct` et la sortie du job sont archivés même en cas d'échec. On peut aller plus loin avec `sbatch --test-only` pour estimer le démarrage avant de soumettre, une partition ou une QoS réservée au CI, et une alerte au-delà d'un seuil d'attente. Sur un cluster cloud dont les nœuds sont créés à la demande, une partie de l'attente vient en outre du démarrage des VM, que `sinfo` signale par un suffixe sur l'état des nœuds.

5.4.3 Le nightly échoue à 2 h du matin : que se demander avant de relancer ?

Trois questions, dans l'ordre :

1. **Quel type d'échec ?** La raison donnée par GitLab (`script_failure` contre `runner_system_failure`, `runner_interrupted` ou timeout) et l'état Slurm (`FAILED` avec un `ExitCode`, contre `NODE_FAIL`, `BOOT_FAIL`, `PREEMPTED` ; `TIMEOUT` de Slurm contre timeout de GitLab). Notre script y répond directement : code 1 ou code 75.
2. **Ai-je les logs ?** Les artefacts conservés même en cas d'échec (`when: always`) : `ci-JOBID.out`, `sacct-JOBID.txt` et le rapport de tests, puis `sacct -j JOBID --format=ALL` pour la mémoire et le nœud. Sans logs, c'est l'observabilité qu'il faut réparer avant de relancer.

3. **Qu'est-ce qui a changé depuis le dernier nightly réussi ?** Commits fusionnés sur `main`, différences du `spack.lock`, maintenance du cluster, quota ou crédits GCP épuisés. Si rien n'a changé dans le dépôt, l'infrastructure est la première suspecte.

On peut en ajouter une quatrième : quelqu'un doit-il vraiment se lever ? Un nightly en échec bloque rarement une version ; les alertes immédiates se réservent à `main` et aux tags.

Un échec de code est déterministe et lié à un commit : test en échec, `FAILED` avec un `ExitCode` non nul, assertion, segfault dans notre code. Le relancer ne sert à rien, masque les tests instables et consomme des heures de calcul : il faut corriger. Un échec d'infrastructure est transitoire et indépendant du code : runner perdu, nœud défaillant, VM impossible à créer (quota ou ressources de la zone épuisés), préemption, NFS. Une relance automatique, limitée à une, est alors légitime. D'où notre configuration : `retry` avec `max: 1`, `when: [runner_system_failure, runner_interrupted]` et `exit_codes: 75`. La documentation récente indique que GitLab 19.1 a séparé de `runner_system_failure` les cas `runner_interrupted` et `runner_external_dependency_failure` : ne lister que l'ancienne valeur ne couvre plus un runner redémarré. Enfin, Slurm remet lui-même en file un job après `NODE_FAIL` : il faut l'annuler avant que le CI ne relance, sinon le test tourne deux fois.

5.5 Erreurs rencontrées et résolution

Tableau 10. Erreurs de la mission 3.

Symptôme	Cause	Résolution
`Could not establish a configuration source`	Slurm installé sans `slurm.conf`	écriture de `slurm.conf` et `slurmdbd.conf`, démarrage des démons
Segfault dans `mca_btl_vader` sur 2 nœuds	émulation : deux nœuds Slurm sur une même machine	1 nœud : OK ; 2 nœuds : `OMPI_MCA_btl=self,tcp` (mini-cluster seulement)
Rangs survivants bloqués 5 min	`srun` laisse tourner les autres tâches	`--kill-on-bad-exit=1` : échec en 1 s
`OMPI was not built with SLURM's PMI support`	`srun` sans plugin PMI	`srun --mpi=pmix`
`sacct` : `JobName=allocation`, nœuds incomplets	enregistrement asynchrone dans `slurmdbd`	attente de 5 s avant l'archivage
Suivi bloqué sur `INCONNU` pendant 11 min (listing 20)	base MariaDB, donc `slurmdbd`, arrêtée	repli sur `scontrol show job`, code 75 après six tours sans état
Job relancé par Slurm après `NODE_FAIL`	remise en file automatique par défaut	`scancel` en `after_script`
`after_script` non exécuté après un timeout (documentation)	le timeout global coupe aussi `after_script`	40 min + 3 min < `timeout` de 45 min
Jobs `tags: [hpc-login]` en attente sans fin sur GitLab.com (anticipé)	aucun runner ne porte ce tag	jobs créés seulement si `HPC_ENABLED` vaut `true`

5.6 Bilan

Ce qui fonctionne : soumission, suivi, verdict classé (0, 1 ou 75) et annulation des jobs orphelins, testés sur un vrai Slurm avec comptabilité, file saturée et panne de nœud.

Ce qui ne fonctionne pas : pas encore d'exécution sur des nœuds réellement distincts ; le transport mémoire partagée n'a pas pu être testé en conditions réelles.

À refaire autrement : tester les cas d'échec (plantage, file pleine, panne) avant le cas nominal : chacun a révélé un défaut du script.

6 Mission 4 : Spack, l'environnement comme code

6.1 Réalisation

Spack est cloné dans le répertoire personnel au tag `v1.2.2` plutôt que sur la branche `develop`, car la version de Spack fait partie de ce qu'il faut figer. `spack compiler find` détecte le GCC 13.3 du système, déclaré comme paquet externe : Spack ne le recompile pas. L'environnement `spack/` demande `openmpi@5 schedulers=slurm %gcc` et `cmake %gcc` (listing 21). Deux réglages se sont imposés à l'expérience. D'une part, une cible générique `x86_64_v3` : `spack arch` donne `icelake` sur notre machine, alors que les VM `e2` reçoivent, selon la disponibilité, un Xeon Haswell, Broadwell ou Skylake ou un AMD EPYC Rome, dont `x86_64_v3` est le dénominateur commun. D'autre part, le Slurm du système est déclaré externe et non constructible : sans cela, `schedulers=slurm` fait compiler par Spack un Slurm complet, avec Python et GLib (76 paquets à construire au lieu de 46).

```yaml
# Environnement Spack du projet mpiflow (Mission 4) : le MANIFESTE (intentions).
# Les versions exactes, variants, compilateurs et hash sont figés dans spack.lock.
#
#   spack env create mpiflow spack/spack.lock   # reproduire l'environnement
#   spack -e mpiflow install
spack:
  specs:
  - openmpi@5 schedulers=slurm %gcc   # MPI lançable par srun (Slurm/PMIx), compilé par le GCC système
  - cmake %gcc
  view: true                          # vue unifiée (bin/, lib/...) dans .spack-env/view
  concretizer:
    unify: true                       # une seule version de chaque dépendance dans l'environnement
  packages:
    all:
      # Cible micro-architecture générique : sans cela Spack optimise pour le CPU
      # de la machine qui concrétise (ici « icelake »). Les VM e2 de GCP peuvent
      # tomber sur un processeur Intel ou AMD : le binaire doit tourner sur les deux.
      target: [x86_64_v3]
    slurm:
      # Slurm est fourni par le cluster : on déclare l'installation système comme
      # « externe » et non constructible. Sans cela, schedulers=slurm fait compiler
      # par Spack un Slurm complet (76 paquets à construire au lieu de 46).
      # À adapter sur le cluster GCP : spack external find --not-buildable slurm
      externals:
      - spec: slurm@23.11.4
        prefix: /usr
      buildable: false
```

Listing 21. Manifeste de l'environnement (`spack/spack.yaml`).

La concrétisation produit 49 specs, dont 46 à construire et 3 externes : GCC, la glibc et Slurm (listing 22). Les deux fichiers sont commités dans `spack/` ; la vue et l'état local (`spack/.spack-env/`) sont ignorés par Git.

```
$ spack -e . concretize --force   # (sortie tronquée à 105 colonnes)
==> Concretized 2 specs:
 -   nxsxsy2  cmake@3.31.11~doc+ncurses+ownlibs~qtgui build_type=Release target=x86_64_v3 %c,cxx=gcc@13.3
 -   5ras4ko      ^compiler-wrapper@1.1.0 target=x86_64_v3
 -   i6kir7u      ^curl@8.20.0~gssapi~ldap~libidn2~librtmp~libssh~libssh2+nghttp2 libs:=shared,static tls
 -   6diukzt          ^nghttp2@1.67.1 target=x86_64_v3 %c,cxx=gcc@13.3.0
```

```
 -   ehec7mr              ^diffutils@3.12 target=x86_64_v3 %c=gcc@13.3.0
 -   x4rtydo          ^openssl@3.6.1~docs+shared certs=mozilla target=x86_64_v3 %c,cxx=gcc@13
    ↪ .3.0
 -   sghkyxj              ^ca-certificates-mozilla@2026-03-19 target=x86_64_v3
[e]  zpevy7h      ^gcc@13.3.0+binutils+bootstrap~graphite+libsanitizer~mold~nvptx~piclibs~
    ↪ profiled~strip
 -   52xg2sk      ^gcc-runtime@13.3.0 target=x86_64_v3
[e]  thy5vqo      ^glibc@2.39 target=x86_64
 -   xbjzygf      ^gmake@4.4.1~guile target=x86_64_v3 %c=gcc@13.3.0
 -   h5ybcvi      ^ncurses@6.6~symlinks+termlib abi=none patches:=7a351bc target=x86_64_v3 %c,
    ↪ cxx=gcc@13.
 -   i5w6x4r      ^zlib-ng@2.3.3+compat+new_strategies+opt+pic+shared target=x86_64_v3 %c,cxx=
    ↪ gcc@13.3.0
 -   iftmwb3  openmpi@5.0.10+atomics~cuda~debug+fortran~gpfs~internal-hwloc~internal-libevent~
    ↪ internal-pm
 -   65abp53      ^autoconf@2.72 target=x86_64_v3
```

Listing 22. Début de la concrétisation (`logs/m4-concretize.txt`) : – à construire, `[e]` externe.

[À compléter : exécution de `scripts/gcp_evidence.sh` : concrétisation sur le cluster cible et durée de l'installation]

6.2 spack.yaml et spack.lock

Tableau 11. Ce que décrivent le manifeste et le verrou.

	`spack.yaml` (manifeste)	`spack.lock` (verrou)
Contenu	des intentions : specs abstraites, contraintes, configuration (cible, externes, vue)	le résultat de la concrétisation : le graphe complet
Versions	une contrainte sur OpenMPI, rien sur les dépendances	toutes : `openmpi@5.0.10`, `pmix@6.1.0`, `cmake@3.31.11`...
Variants	ceux qu'on impose (`schedulers=slurm`)	tous, y compris les valeurs par défaut
Compilateurs	`%gcc`	`gcc@13.3.0`, nœud du graphe (dépendance de compilation)
Identité	–	hash de chaque nœud, arêtes typées (`build`, `link`), OS et cible
Forme	27 lignes de YAML commenté, écrites à la main	JSON généré : 64 Ko, 49 specs

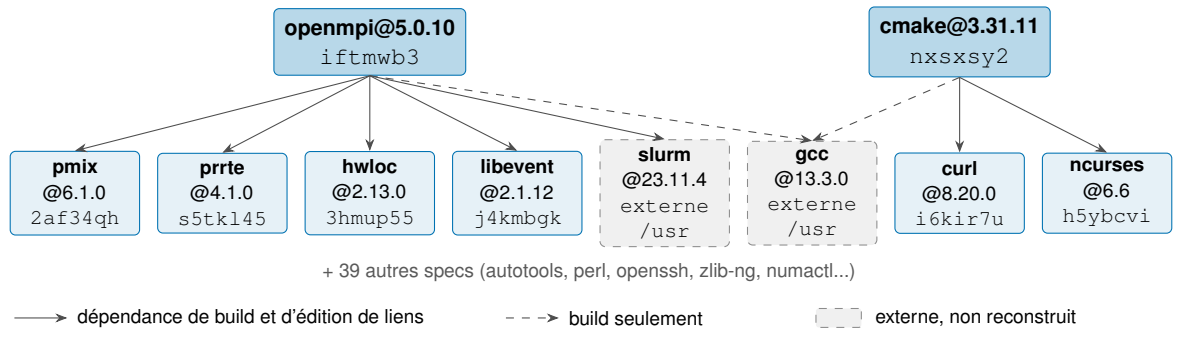


Figure 4. Extrait du graphe enregistré dans `spack.lock` : les deux specs racines et quelques dépendances directes, avec les sept premiers caractères de leur hash. Les externes (GCC, Slurm) sont propres à la machine qui a concrétisé.

Le tableau 11 et la figure 4 résument ce que contient chaque fichier ; le listing 23 montre la spec concrète d'OpenMPI telle qu'elle est enregistrée.

```
# spack.lock (extrait mis en forme ; hash abrégés à 7 caractères) -- 49 specs
"_meta": {"file-type": "spack-lockfile", "lockfile-version": 6, "specfile-version": 5},
"roots": [
  {"hash": "iftmwb3...", "spec": "openmpi@5 schedulers=slurm %gcc"},
  {"hash": "nxsxsy2...", "spec": "cmake %gcc"},
],
"concrete_specs": {
 "iftmwb3...": {
  "name": "openmpi",
  "version": "5.0.10",
  "arch": {"platform": "linux", "platform_os": "ubuntu24.04", "target": "x86_64_v3"},
  "namespace": "builtin",
  "parameters": {"fabrics": ["none"], "fortran": true, "internal-pmix": false, "schedulers":
    ↪ ["slurm"]}, ...
  "dependencies": [
   {"name": "gcc", "hash": "zpevy7h...", "deptypes": ["build"]},
   {"name": "hwloc", "hash": "3hmup55...", "deptypes": ["build", "link"]},
   {"name": "libevent", "hash": "j4kmbgk...", "deptypes": ["build", "link"]},
   {"name": "pmix", "hash": "2af34qh...", "deptypes": ["build", "link"]},
   {"name": "prrte", "hash": "s5tkl45...", "deptypes": ["build", "link"]},
   {"name": "slurm", "hash": "qjjvtor...", "deptypes": ["build", "link"]},
   ... (18 dépendances au total)
  ]
 },
 "qjjvtor...": {"name": "slurm", "version": "23.11.4", "external": {"path": "/usr"}, ...}
}
```

Listing 23. Extrait de `spack.lock`, mis en forme (`logs/m4-lock-excerpt.txt`).

6.3 Questions de cadrage

6.3.1 Syntaxe package@version%compiler+variant~variant et spec de HDF5

`@` fixe une version ou un intervalle (`@1.14.6`, `@1.14:`), `%` le compilateur, `+` active un variant booléen et `~` le désactive ; `clÃl'=valeur` règle un variant à valeurs (`schedulers=slurm`, `target=x86_64_v3`) et `^` contraint une dépendance (`^openmpi@5`). Depuis Spack 1.0, le compilateur est une dépendance comme une autre : la sortie affiche `%c,cxx=gcc@13.3.0`. La spec demandée s'écrit `hdf5+mpi+fortran~cxx %gcc`, ou, entièrement figée, `hdf5@1.14.6+mpi+fortran~cxx %gcc@13.3.0 ^openmpi@5.0.10`. On vérifie avec `spack spec` qu'elle est bien comprise (listing 24). D'après `spack info hdf5`, `cxx` est déjà désactivé par défaut et `mpi` activé : les écrire quand même protège la spec contre un changement de valeur par défaut dans une future version de la recette. On remarque aussi la cible `icelake` hors de l'environnement, contre `x86_64_v3` dans l'environnement.

```
$ spack spec hdf5+mpi+fortran~cxx %gcc@13.3.0   # (extrait)
 -   hdf5@1.14.6~cxx+fortran~hl~ipo~java~map+mpi+shared~subfiling~szip~threadsafe+tools api=
    ↪ default build_type=R
[e]      ^gcc@13.3.0+binutils+bootstrap~graphite+libsanitizer~mold~nvptx~piclibs~profiled~
    ↪ strip build_type=RelWi
 -       ^openmpi@5.0.10+atomics~cuda~debug+fortran~gpfs~internal-hwloc~internal-libevent~
    ↪ internal-pmix~ipv6~jav
$ spack arch
linux-ubuntu24.04-icelake
```

Listing 24. Vérification de la spec par `spack spec` (`logs/m4-hdf5.txt`).

6.3.2 Pourquoi spack.lock suffit-il à reproduire l'environnement, et pas spack.yaml seul ?

`spack.yaml` décrit des intentions, et le résultat de leur concrétisation dépend du contexte : version de Spack et des recettes (les versions préférées évoluent : aujourd'hui OpenMPI 5.0.10 et PMIx 6.1.0, autre chose dans six mois), compilateurs disponibles, paquets externes déclarés,

micro-architecture de la machine (`icelake` ici si l'on ne force pas la cible), réglages du solveur. Deux machines, ou deux dates, donnent deux environnements différents. `spack.lock` fige au contraire le graphe complet : versions, variants, compilateurs, cible, arêtes et hash. `spack env create mpiflow spack/spack.lock` suivi de `spack install` réinstalle exactement ce graphe sans relancer le solveur, et le hash identifie la configuration : même hash, même build, même entrée de buildcache.

Le verrou a toutefois des limites, que l'exercice fait apparaître. Il suppose que la version de Spack sait encore relire ses recettes (d'où le tag `v1.2.2`) et que les sources restent téléchargeables (leurs sommes de contrôle sont vérifiées). Surtout, il contient des éléments propres à la machine : l'OS (`ubuntu24.04`) et les externes (GCC et Slurm 23.11.4 dans `/usr`, figure 4). Notre verrou local démontre le mécanisme, mais le verrou de référence doit être produit sur le cluster cible, dont le nœud de connexion et les nœuds de calcul partagent la même image, puis commité. On en déduit qu'il est reproductible « à machine identique », pas sur n'importe quel poste.

6.3.3 Quel est l'intérêt du buildcache pour un pipeline qui installe l'environnement à chaque exécution ?

Un job CI part d'un environnement vierge : `spack install` y recompilerait les 46 paquets de notre environnement (OpenMPI, PMIx, PRRTE, hwloc, CMake, autotools...) à chaque pipeline, soit des dizaines de minutes à quelques heures sur 2 vCPU, bien au-delà de notre `timeout` de 45 min. Un buildcache stocke les binaires indexés par leur hash : l'installation devient un téléchargement suivi d'une relocalisation, en quelques minutes. Comme les hash du verrou sont stables, le cache est atteint exactement, et seuls les nœuds modifiés se recompilent. Le pipeline de `main` peut publier les nouveaux binaires (`spack buildcache push`), signés par la clé GPG du projet (variable `SPACK_GPG_KEY`, de type *File* et protégée), dans un registre OCI (le registre de conteneurs du projet GitLab) ou dans un bucket GCS. Le cluster et le CI installent alors les mêmes binaires. Par défaut, Spack ne connaît qu'un miroir de sources (`spack-public [s]`) : un buildcache se configure explicitement.

6.4 Erreurs rencontrées et résolution

Tableau 12. Erreurs de la mission 4.

Symptôme	Cause	Résolution
Échec de l'amorçage de `clingo` (`ghcr.io` injoignable depuis notre réseau)	Spack télécharge son solveur au premier `spack spec`	`pip install clingo` : Spack utilise un clingo importable par son Python
76 paquets à construire, dont `slurm@25-05-1-1`, Python, GLib	`schedulers=slurm` ajoute Slurm comme dépendance	Slurm du système déclaré externe et non constructible : 46 paquets
`spack external find slurm` : `Could not establish a configuration source`	la détection lance `salloc --version`, qui échoue sans `slurm.conf`	détection relancée une fois le mini-cluster configuré
Commentaires de `spack.yaml` effacés	`spack external find` réécrit le manifeste	manifeste réécrit à la main
Cible `icelake` détectée	concrétisation pour le processeur de la machine	`packages: all: target: [x86_64_v3]`
`spack install` impossible en local	pas d'accès aux miroirs de sources	installation sur le nœud de connexion GCP ([À compléter : durée])

6.5 Bilan

Ce qui fonctionne : environnement concrétisé (49 specs) avec une cible portable et Slurm externe ; `spack.yaml` et `spack.lock` commités ; spec de HDF5 vérifiée.

Ce qui ne fonctionne pas : installation, et build du test avec l'OpenMPI de Spack, pas encore faits (sur GCP).

À refaire autrement : concrétiser directement sur le nœud de connexion, et mettre en place le buildcache avant d'appeler `spack install` depuis le CI.

7 Conclusion et auto-diagnostic

La séance répond à la question qui la sous-tend : un pipeline HPC ne se contente pas de transposer le CI du cloud. Les tests rapides tournent sur des runners partagés, mais ce qu'ils valident s'arrête à la logique du code ; l'intégration avec le MPI, le lanceur et le réseau du cluster passe par Slurm, avec deux horloges à gérer, des jobs orphelins à annuler et des échecs à classer avant de relancer. Spack apporte la pièce manquante, un environnement versionné à côté du code, à condition de le concrétiser sur la machine cible.

Plusieurs réponses aux questions de cadrage sont sorties nuancées de l'expérience : oublier `needs:` ne casse pas un pipeline GitLab, le test MPI des diapositives passe toujours en `Release`, et un rang qui plante ne fait pas échouer un job Slurm avant le wall-time. Les principales difficultés ont été techniques : les deux nœuds émulés sur une même machine ont fait planter le transport mémoire partagée d'Open MPI, et la comptabilité asynchrone de Slurm renvoyait des informations incomplètes juste après la fin d'un job.

7.1 Acquis

- La sémantique réelle d'un pipeline : stages et DAG (`needs`), transmission des artefacts, types de pipelines et `workflow:rules`, conditions avec `rules`.
- Les trois protections d'un secret (masqué, protégé, type *File*), ce qu'aucune ne garantit seule, et le passage par l'entrée standard.
- L'orchestration du CI vers Slurm : `timeout` et wall-time, états terminaux, annulation des jobs orphelins, distinction entre échec de code et échec d'infrastructure.
- Le manifeste et le verrou de Spack, les paquets externes et la cible micro-architecture.
- Une méthode : provoquer volontairement chaque échec avant de faire confiance au cas nominal.

7.2 Points à retravailler

- L'enregistrement et la sécurisation d'un runner sur le cluster GCP (utilisateur dédié, runner protégé, `/home` partagé).
- Le buildcache de Spack en pratique : publication, signature GPG, utilisation depuis le CI.
- La génération de modules Lmod depuis Spack (`spack module lmod refresh`).
- Le runner auto-hébergé de GitHub Actions et les conteneurs Apptainer (cours suivant).

7.3 Utilisation dans un projet d'imagerie médicale

Prenons un code de segmentation 3D (ITK, MPI ou GPU) appliqué à des volumes IRM ou scanner.

- **À chaque push :** lint, tests unitaires et un test de bout en bout sur un petit fantôme synthétique ou un volume anonymisé de quelques mégaoctets, dans un conteneur.

- **Chaque nuit, via Slurm :** segmentation d'un jeu de référence ; le test compare le coefficient de Dice à une segmentation de référence avec une tolérance, plutôt qu'une égalité bit à bit, car les réductions MPI et GPU ne sont pas déterministes au bit près.
- **Traçabilité :** chaque résultat archivé avec le SHA du commit, le hash du `spack.lock` (ITK, VTK, HDF5, PyTorch), le JobID Slurm et les sommes SHA-256 des volumes produits, ce qui permet de rejouer une figure de publication.
- **Données de santé :** aucune donnée de patient dans les artefacts, les logs ou les caches du CI (RGPD) ; seulement des données anonymisées ou synthétiques, un accès au stockage par variable protégée, et les jobs réels uniquement sur le cluster habilité.

Références

1. L. A. Steffenel, *Virtualisation et cloud pour le HPC, Cours 6 : DevOps*, M2 CHPS, Université de Reims Champagne-Ardenne, 2026.
2. GitLab, *CI/CD YAML syntax reference* (`needs`, `retry`, `after_script`, `timeout`), `https://docs.gitlab.com/ci/yaml/`.
3. GitLab, *Job artifacts*, `https://docs.gitlab.com/ci/jobs/job_artifacts.html`.
4. GitLab, *Specify when jobs run with rules*, `https://docs.gitlab.com/ci/jobs/job_rules/`.
5. GitLab, *Deprecated keywords* (`only`, `except`), `https://docs.gitlab.com/ci/yaml/deprecated_keywords/`.
6. GitLab, *CI/CD variables*, `https://docs.gitlab.com/ci/variables/`.
7. GitLab, *Configuring runners*, `https://docs.gitlab.com/ci/runners/configure_runners/`.
8. Spack, documentation de la version 1.x, `https://spack.readthedocs.io/`.
9. Google Cloud, annonce des machines E2, `https://cloud.google.com/blog/products/compute/google-compute-engine-gets-new-e2-vm-machine-types`.
10. GitHub, `actions/upload-artifact`, `https://github.com/actions/upload-artifact`.
11. gitlab-ci-local, `https://github.com/firecow/gitlab-ci-local`.

Annexe A : pipeline *.gitlab-ci.yml*

Ce fichier définit le pipeline complet de la figure 1 : jobs des missions 1 et 2 pour les runners partagés, et jobs Slurm de la mission 3 pour le runner du nœud de connexion.

```
# .gitlab-ci.yml -- projet mpiflow (M2 CHPS, CHPS0902, Cours 6 : séance inversée)
#
#   Mission 1 : build -> test (artifacts + needs)
#   Mission 2 : lint (MR uniquement), package (tags vX.Y.Z), secret (--password-stdin)
#   Mission 3 : test_slurm (soumission + polling), test_slurm_wait (sbatch --wait),
#               test_slurm_ssh (runner partagé -> nœud de connexion en SSH)
#
# Variables à définir dans Settings > CI/CD > Variables :
#   FAKE_REGISTRY_TOKEN  Variable, Masked, Protected  (fausse valeur, >= 8 caractères)
#   HPC_ENABLED          "true" quand un runner « hpc-login » est enregistré sur le cluster
#   SLURM_SSH_KEY        File, Protected   } uniquement pour la variante SSH
#   SLURM_KNOWN_HOSTS    File              }
#   SLURM_LOGIN_HOST, SLURM_USER           }

# Pipelines de MR quand une MR est ouverte, pipelines de branche sinon : pas de
# doublons (un job sans rules vaut except: merge_requests, cf. doc « job rules »).
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH            # push sur une branche, pipelines planifiés
    - if: $CI_COMMIT_TAG               # releases

stages: [lint, build, test, package, test_hpc]

variables:
  DEBIAN_FRONTEND: noninteractive
  # Open MPI dans Docker : ptrace entre processus interdit -> pas de copie CMA
  OMPI_MCA_btl_vader_single_copy_mechanism: none

default:
  image: gcc:13-bookworm
  interruptible: true                  # un nouveau push annule le pipeline devenu obsolète

# ------------------------------------------------------------------ modèles -----
.apt_mpi:
  before_script:
    - apt-get update -qq
    - apt-get install -y -qq cmake libopenmpi-dev openmpi-bin > /dev/null

.every_pipeline:
  rules:
    - when: on_success                 # branche, MR et tag (le workflow filtre le reste)

# ========================================================= Mission 1 ============
build:
  stage: build
  extends: [.apt_mpi, .every_pipeline]
  script:
    - cmake -B build -DCMAKE_BUILD_TYPE=Release
        -DMPIEXEC_PREFLAGS="--allow-run-as-root;--oversubscribe"
    - cmake --build build -j"$(nproc)"
  artifacts:
    paths: [build/]
    expire_in: 1 week

test:
  stage: test
  extends: [.apt_mpi, .every_pipeline]
  needs: [build]                       # démarre dès la fin de build, récupère build/
  script:
    - cd build
    - ctest --output-on-failure --output-junit junit.xml
    # root dans le conteneur + 2 à 4 cœurs sur le runner : les deux options sont requises
    - mpirun --allow-run-as-root --oversubscribe -np 4 tests/test_mpi
  artifacts:
```

```yaml
    when: always                        # le rapport JUnit est utile surtout en cas d'échec
    reports:
      junit: build/junit.xml

# ========================================================= Mission 2 ============
lint:
  stage: lint
  needs: []
  before_script:
    - apt-get update -qq && apt-get install -y -qq cppcheck > /dev/null
  script:
    - cppcheck --version
    - cppcheck --enable=warning,style,performance,portability --std=c11
        --error-exitcode=1 --inline-suppr --suppress=missingIncludeSystem tests/
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

package:
  stage: package
  needs:
    - build
    - job: test                         # sans cela, needs: [build] seul publierait
      artifacts: false                  # un paquet même si les tests échouent
  variables:
    PKG: mpiflow-${CI_COMMIT_TAG}-linux-x86_64
  script:
    # tar explicite plutôt que cpack : l'arbre de build CMake contient des chemins
    # absolus (non relocalisable) si le job package ne tourne pas au même chemin
    - mkdir -p "dist/$PKG/bin"
    - cp build/tests/test_mpi "dist/$PKG/bin/"
    - cp README.md "dist/$PKG/"
    - printf 'projet=%s\ntag=%s\ncommit=%s\npipeline=%s\ndate=%s\n'
        "$CI_PROJECT_PATH" "$CI_COMMIT_TAG" "$CI_COMMIT_SHA" "$CI_PIPELINE_URL"
        "$(date -u +%FT%TZ)" > "dist/$PKG/BUILDINFO.txt"
    - tar -C dist -czf "dist/$PKG.tar.gz" "$PKG"
    - cd dist && sha256sum "$PKG.tar.gz" | tee checksums.txt   # BUILDINFO est dans le tarball
  artifacts:
    paths: [dist/*.tar.gz, dist/checksums.txt]
    expire_in: 6 months
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/

registry_login:
  stage: package
  needs: []
  extends: .every_pipeline
  script:
    - echo "ref=$CI_COMMIT_REF_NAME protégée=$CI_COMMIT_REF_PROTECTED"
    - |
      if [ -z "${FAKE_REGISTRY_TOKEN:-}" ]; then
        echo "FAKE_REGISTRY_TOKEN vide : variable Protected sur une ref non protégée."
        exit 0
      fi
    - echo "Test du masquage -> $FAKE_REGISTRY_TOKEN"    # doit s'afficher [MASKED]
    - echo "$FAKE_REGISTRY_TOKEN" |
        ./scripts/fake_docker_login.sh -u ci-user --password-stdin registry.example.org
    # Version réelle (image docker:cli + service docker:dind) :
    #   echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin "
    ↪ $CI_REGISTRY"

# ========================================================= Mission 3 ============
# Runner « shell » installé sur le nœud de connexion du cluster (tag hpc-login) :
# il n'ouvre aucune connexion entrante (il interroge GitLab en HTTPS) et partage
# /home (NFS) avec les nœuds de calcul. Il orchestre ; le calcul passe par Slurm.
.hpc:
  stage: test_hpc
  tags: [hpc-login]
  needs:                                # attendre les tests rapides : inutile de consommer
    - job: test                         # des heures cluster si le CI est déjà rouge...
      artifacts: false                  # ...mais sans son binaire (Debian/Open MPI 4.1),
                                        # inutilisable sur le cluster : on recompile
  variables:
    POLL_INTERVAL: "30"
```

```
    SLURM_OPTS: ""                      # ex. "--partition=compute" (vérifier avec sinfo)
    # script coupé à 40 min pour laisser à after_script le temps d'annuler le job
    RUNNER_SCRIPT_TIMEOUT: 40m
    RUNNER_AFTER_SCRIPT_TIMEOUT: 3m
  before_script:
    # environnement installé une fois sur le nœud de connexion (store Spack partagé) :
    # ici « spack install » ne recompile rien, il (re)crée seulement la vue du dépôt
    - source "${SPACK_ROOT:-$HOME/spack}/share/spack/setup-env.sh"
    - spack -e spack/ install --fail-fast
    - spack env activate -d spack/
    - cmake -B build-hpc -DCMAKE_BUILD_TYPE=Release
    - cmake --build build-hpc -j4
  after_script:
    # exécuté aussi si le job est annulé ou coupé : pas de job Slurm orphelin
    - |
      if [ -s .slurm_jobid ] && squeue -h -j "$(cat .slurm_jobid)" 2>/dev/null | grep -q .;
    ↪ then
        scancel "$(cat .slurm_jobid)" && echo "Job Slurm $(cat .slurm_jobid) annulé"
      fi
  artifacts:
    when: always                        # les logs sont la seule fenêtre sur un échec nocturne
    paths: [ci-*.out, sacct-*.txt]
    expire_in: 1 week
  rules:
    - if: $HPC_ENABLED != "true"        # sans runner hpc-login, le job resterait « pending »
      when: never
    - if: $CI_PIPELINE_SOURCE == "schedule"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
      allow_failure: true

test_slurm:
  extends: .hpc
  timeout: 45m                          # > wall-time Slurm (5 min) + attente en file
  retry:
    max: 1
    when: [runner_system_failure, runner_interrupted]
    exit_codes: 75                      # NODE_FAIL / BOOT_FAIL / PREEMPTED (submit_and_poll.sh)
    ↪
  script:
    - ./slurm/submit_and_poll.sh slurm/job_ci.sh $SLURM_OPTS

test_slurm_wait:
  extends: .hpc
  timeout: 45m
  script:
    - sbatch --wait --parsable $SLURM_OPTS slurm/job_ci.sh > .slurm_jobid
    - cat "ci-$(cat .slurm_jobid).out"

# Variante sans runner sur le cluster : un runner partagé de GitLab.com se connecte
# en SSH à l'IP publique du nœud de connexion (clé en variable File, Protected).
test_slurm_ssh:
  stage: test_hpc
  image: debian:bookworm-slim
  needs: []
  timeout: 45m
  variables:
    REMOTE: "$SLURM_USER@$SLURM_LOGIN_HOST"
    WORKDIR: "ci/$CI_PROJECT_NAME/$CI_PIPELINE_ID"
  before_script:
    - apt-get update -qq && apt-get install -y -qq openssh-client rsync > /dev/null
    - chmod 600 "$SLURM_SSH_KEY"        # variable File : elle contient un CHEMIN
    - install -d -m 700 ~/.ssh && cp "$SLURM_KNOWN_HOSTS" ~/.ssh/known_hosts
  script:
    - rsync -a --delete -e "ssh -i $SLURM_SSH_KEY -o BatchMode=yes" ./ "$REMOTE:$WORKDIR/"
    - ssh -i "$SLURM_SSH_KEY" -o BatchMode=yes -o ServerAliveInterval=60 "$REMOTE"
        "cd $WORKDIR && bash -l slurm/ci_remote.sh"
  after_script:                         # shell séparé : seules les variables CI sont connues
    - rsync -a -e "ssh -i $SLURM_SSH_KEY -o BatchMode=yes"
        --include='ci-*.out' --include='sacct-*.txt' --exclude='*' "$REMOTE:$WORKDIR/" . ||
    ↪ true
    - ssh -i "$SLURM_SSH_KEY" -o BatchMode=yes "$REMOTE"
        "cd $WORKDIR && [ -s .slurm_jobid ] && scancel \$(cat .slurm_jobid)" || true
```

```
  artifacts:
    when: always
    paths: [ci-*.out, sacct-*.txt]
    expire_in: 1 week
  rules:
    - if: $SLURM_SSH_KEY == null || $SLURM_LOGIN_HOST == null
      when: never
    - if: $CI_PIPELINE_SOURCE == "schedule"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
      allow_failure: true
```

Annexe B : scripts Slurm *job_ci.sh* et *submit_and_poll.sh*

Le premier script est le job soumis à Slurm ; le second le soumet, suit son état et rend un verdict exploitable par GitLab. Ils s'exécutent depuis la racine du dépôt.

```
#!/bin/bash
# job_ci.sh -- test MPI multi-nœuds soumis par le pipeline (Mission 3)
#
# Ressources : 2 nœuds × 2 tâches = 4 rangs MPI (adapté à des nœuds e2-standard-2,
# 2 vCPU). Sur un vrai cluster : --ntasks=8 --ntasks-per-node=4.
# Toutes les valeurs #SBATCH peuvent être surchargées sur la ligne de commande
# (sbatch --partition=compute job_ci.sh) : vérifier le nom de partition avec sinfo.
#
#SBATCH --job-name=ci-mpiflow
#SBATCH --partition=debug
#SBATCH --nodes=2
#SBATCH --ntasks=4
#SBATCH --ntasks-per-node=2
#SBATCH --time=00:05:00
#SBATCH --output=ci-%j.out

set -uo pipefail

echo "== Job ${SLURM_JOB_ID} : ${SLURM_NNODES} nœud(s) [${SLURM_JOB_NODELIST}], ${SLURM_NTASKS}
    ↪ tâches"
echo "== Soumis depuis ${SLURM_SUBMIT_DIR} (commit ${CI_COMMIT_SHORT_SHA:-hors CI})"
cd "${SLURM_SUBMIT_DIR}" || exit 2

# --- 1. Environnement logiciel -----------------------------------------------
# Ordre de préférence : modules Lmod > environnement Spack versionné > système.
# Sur le cluster GCP du Cours 5, Lmod n'est pas configuré : on active directement
# l'environnement Spack du dépôt (spack/spack.yaml + spack.lock), installé au
# préalable sur le système de fichiers partagé (/home est monté en NFS).
SPACK_ROOT="${SPACK_ROOT:-$HOME/spack}"
if type module >/dev/null 2>&1 && module -t avail openmpi 2>&1 | grep -q '^openmpi'; then
    module purge
    module load openmpi
    ENV_SOURCE="Lmod (module load openmpi)"
elif [[ -f "${SPACK_ROOT}/share/spack/setup-env.sh" && -f spack/spack.lock ]]; then
    # shellcheck disable=SC1091
    source "${SPACK_ROOT}/share/spack/setup-env.sh"
    spack env activate -d spack/
    ENV_SOURCE="Spack (environnement du dépôt, spack.lock)"
else
    ENV_SOURCE="système (PATH par défaut)"
fi
echo "== Environnement : ${ENV_SOURCE}"
echo "== mpicc : $(command -v mpicc || echo absent)"

# --- 2. Exécution ------------------------------------------------------------
BIN="${MPIFLOW_BIN:-build-hpc/tests/test_mpi}"
if [[ ! -x "${BIN}" ]]; then
    echo "FAILED: binaire ${BIN} introuvable (compilé sur le nœud de connexion ?)"
    exit 2
fi

# EXPECTED_RANKS : le test échoue si MPI_COMM_WORLD n'a pas la taille demandée,
# ce qui arrive quand srun lance des « singletons » faute de support PMI.
# --kill-on-bad-exit=1 : si un rang plante, srun tue les autres immédiatement
# (sinon ils restent bloqués dans une collective jusqu'au wall-time).
export EXPECTED_RANKS="${SLURM_NTASKS}"
srun --mpi="${SRUN_MPI:-pmix}" --kill-on-bad-exit=1 "${BIN}"
rc=$?

# --- 3. Verdict lisible par le pipeline --------------------------------------
if [[ ${rc} -eq 0 ]]; then
    echo "SUCCESS: MPI test passed (${SLURM_NTASKS} rangs sur ${SLURM_NNODES} nœuds)"
else
    echo "FAILED: srun a renvoyé ${rc}"
fi
exit "${rc}"
```

slurm/job_ci.sh

```
#!/bin/bash
# submit_and_poll.sh -- soumet un job Slurm et suit son état jusqu'au bout (Mission 3)
#
# usage : slurm/submit_and_poll.sh <script.sh> [options sbatch...]
# env   : POLL_INTERVAL (s, défaut 30)
#
# Codes retour (lus par le job GitLab) :
#   0  job COMPLETED avec ExitCode 0
#   1  échec de code (FAILED, TIMEOUT Slurm, OUT_OF_MEMORY, CANCELLED...)
#   75 échec d'infrastructure (NODE_FAIL, BOOT_FAIL, PREEMPTED, état introuvable)
#      -> relançable
set -uo pipefail

script="$1"; shift
interval="${POLL_INTERVAL:-30}"

# --parsable : sortie « jobid » ou « jobid;cluster », sans phrase à analyser
jobid="$(sbatch --parsable "$@" "${script}")" || { echo "sbatch a échoué" >&2; exit 1; }
jobid="${jobid%%;*}"
echo "${jobid}" > .slurm_jobid          # relu par after_script pour annuler le job
echo "Job Slurm ${jobid} soumis ($(date -Is))"

# état d'un champ du job sans passer par la base : scontrol garde les jobs terminés
# MinJobAge secondes (300 par défaut)
scontrol_field() { scontrol show job -o "${jobid}" 2>/dev/null | grep -o "$1=[^ ]*" | cut -d= -
    ↪ f2; }

start=$(date +%s)
state=""
shown_start=""
unknown=0
max_unknown="${MAX_UNKNOWN:-6}"
while true; do
    # -X : ligne de l'allocation seulement (pas les steps) ; -P : sortie séparée par « | »
    state="$(sacct -X -n -P -j "${jobid}" -o State 2>/dev/null | head -1)"
    state="${state%% *}"                # « CANCELLED by 1000 » -> « CANCELLED »
    # sacct peut être vide juste après la soumission (slurmdbd pas encore à jour)
    [[ -z "${state}" ]] && state="$(squeue -h -j "${jobid}" -o %T 2>/dev/null)"
    # base de comptabilité indisponible et job sorti de la file : on interroge le contrôleur
    [[ -z "${state}" ]] && state="$(scontrol_field JobState)"
    reason="$(squeue -h -j "${jobid}" -o %r 2>/dev/null)"
    printf '[%s] +%5ss  job %s  %-10s %s\n' "$(date +%T)" "$(( $(date +%s) - start ))" \
           "${jobid}" "${state:-INCONNU}" "${reason:+raison=${reason}}"

    case "${state}" in
        PENDING)
            # la raison vaut « None » tant que l'ordonnanceur n'a pas évalué le job
            if [[ -z "${shown_start}" && "${reason}" != "None" ]]; then
                squeue -h --start -j "${jobid}" -o '   début estimé par Slurm : %S'
                shown_start=1
            fi ;;
        COMPLETED|FAILED|TIMEOUT|CANCELLED|OUT_OF_MEMORY|NODE_FAIL|BOOT_FAIL|PREEMPTED|
    ↪ DEADLINE)
            break ;;
        "")
            # ni sacct, ni squeue, ni scontrol : inutile d'attendre le timeout du CI
            unknown=$((unknown + 1))
            if (( unknown >= max_unknown )); then
                echo "Verdict : état du job introuvable après ${unknown} tentatives (
    ↪ comptabilité indisponible ?)" >&2
                exit 75
            fi ;;
        *) unknown=0 ;;
    esac
    sleep "${interval}"
done

# la comptabilité (slurmdbd) est asynchrone : juste après la fin du job, sacct peut
```

```
# encore afficher JobName=allocation et une liste de nœuds incomplète
sleep "${SACCT_SETTLE:-5}"
echo "---- sacct ----"
sacct -j "${jobid}" -o JobID,JobName%14,State,ExitCode,Elapsed,NNodes,NodeList%16 2>&1 | tee "
    ↪ sacct-${jobid}.txt"
echo "---- sortie du job ----"
cat "ci-${jobid}.out" 2>/dev/null || echo "(pas de fichier ci-${jobid}.out)"

exitcode="$(sacct -X -n -P -j "${jobid}" -o ExitCode 2>/dev/null | head -1)"   # « code:signal
    ↪ »
[[ -z "${exitcode}" ]] && exitcode="$(scontrol_field ExitCode)"                 # repli sans
    ↪ base
case "${state}" in
    COMPLETED)
        if [[ "${exitcode%%:*}" == 0 ]] && grep -q '^SUCCESS' "ci-${jobid}.out"; then
            echo "Verdict : succès"; exit 0
        fi
        echo "Verdict : COMPLETED mais ExitCode=${exitcode} ou SUCCESS absent"; exit 1 ;;
    NODE_FAIL|BOOT_FAIL|PREEMPTED)
        echo "Verdict : échec d'INFRASTRUCTURE (${state}) -> relance possible" >&2; exit 75 ;;
    *)
        echo "Verdict : échec du JOB (${state}, ExitCode=${exitcode})" >&2; exit 1 ;;
esac
```

slurm/submit_and_poll.sh

Annexe C : test MPI *test_mpi.c*

Le test est compilé par CMake et lancé par CTest (à 1 et 4 rangs), par `mpirun` dans le conteneur CI et par `srun` sur le cluster.

```c
/*
 * test_mpi.c -- test d'intégration MPI minimal (projet mpiflow, Cours 6 CHPS0902)
 *
 * Vérifie :
 *   1. MPI_Bcast   : la valeur 42 diffusée par le rang 0 arrive sur tous les rangs ;
 *   2. MPI_Allreduce : chaque rang participe (somme des rangs = n(n-1)/2) ;
 *   3. la taille du communicateur : si EXPECTED_RANKS est défini (CTest, Slurm),
 *      MPI_COMM_WORLD doit contenir exactement ce nombre de rangs. Cela détecte
 *      un lancement en « singletons » (srun sans support PMI : N programmes
 *      indépendants de taille 1 qui réussissent tous... à tort).
 *
 * Les vérifications n'utilisent PAS assert() : en CMAKE_BUILD_TYPE=Release,
 * -DNDEBUG supprime les assert() et le test passerait toujours.
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int errors = 0;

    /* 1. Broadcast depuis le rang 0 */
    int data = (rank == 0) ? 42 : 0;
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (data != 42) {
        fprintf(stderr, "[rang %d] MPI_Bcast : reçu %d au lieu de 42\n", rank, data);
        errors++;
    }

    /* 2. Allreduce : tous les rangs participent */
    long sum = 0, mine = rank;
    MPI_Allreduce(&mine, &sum, 1, MPI_LONG, MPI_SUM, MPI_COMM_WORLD);
    long expected_sum = (long)size * (size - 1) / 2;
    if (sum != expected_sum) {
        fprintf(stderr, "[rang %d] MPI_Allreduce : %ld au lieu de %ld\n", rank, sum,
    ↪ expected_sum);
        errors++;
    }

    /* 3. Taille attendue du communicateur */
    const char *env = getenv("EXPECTED_RANKS");
    if (env != NULL && atoi(env) > 0 && atoi(env) != size) {
        fprintf(stderr, "[rang %d] taille de MPI_COMM_WORLD = %d, attendu %s "
                        "(lancement sans PMI ?)\n", rank, size, env);
        errors++;
    }

    /* Placement des rangs : utile pour prouver une exécution multi-nœuds */
    char host[MPI_MAX_PROCESSOR_NAME] = {0};
    int len;
    MPI_Get_processor_name(host, &len);
    char *hosts = NULL;
    if (rank == 0)
        hosts = malloc((size_t)size * MPI_MAX_PROCESSOR_NAME);
    MPI_Gather(host, MPI_MAX_PROCESSOR_NAME, MPI_CHAR,
               hosts, MPI_MAX_PROCESSOR_NAME, MPI_CHAR, 0, MPI_COMM_WORLD);

    int total_errors = 0;
    MPI_Allreduce(&errors, &total_errors, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

```
67
68      if (rank == 0) {
69          int nhosts = 0;
70          for (int r = 0; r < size; r++) {
71              const char *h = hosts + (size_t)r * MPI_MAX_PROCESSOR_NAME;
72              int seen = 0;
73              for (int q = 0; q < r && !seen; q++)
74                  seen = !strcmp(h, hosts + (size_t)q * MPI_MAX_PROCESSOR_NAME);
75              nhosts += !seen;
76              printf("  rang %d sur %s\n", r, h);
77          }
78          free(hosts);
79          if (total_errors == 0)
80              printf("MPI test passed with %d ranks on %d host(s)\n", size, nhosts);
81          else
82              printf("MPI test FAILED: %d error(s) on %d ranks\n", total_errors, size);
83      }
84
85      MPI_Finalize();
86      return total_errors == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
87  }
```